МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»
Навчально-науковий інститут телекомунікаційних систем
Кафедра _____

ЗВІТ

з лабораторної роботи № 3

з дисципліни «Розробка баз даних»

Тема: «Знайомство з мовою SQL. Створення, редагування і видалення баз даних та таблиць»

Предметна область: «STEM-студія для дітей "Стемік"»

(система обліку занять, відвідувань, оплат і 3D-друку)

Виконали:
студенти групи _____
бригада:
Левченко __. __.
Гвоздецька __. __.
Ігнатенко __. __.

Перевірив(ла):
_____
Оцінка: _____ Підпис: _____

Київ – 2026

ЗМІСТ

ЛАБОРАТОРНА РОБОТА № 3. ЗНАЙОМСТВО З МОВОЮ SQL. СТВОРЕННЯ, РЕДАГУВАННЯ І ВИДАЛЕННЯ БАЗ ДАНИХ ТА ТАБЛИЦЬ 3

3.1 Підмови SQL, використані в роботі..... 3

3.2 Створення бази даних (CREATE DATABASE)..... 4

3.3 Створення таблиць (CREATE TABLE)..... 5

3.4 Наповнення таблиць даними (INSERT)..... 8

3.5 Зміна структури таблиць (ALTER TABLE)..... 11

3.6 Оновлення і видалення записів (UPDATE, DELETE)..... 13

3.7 TRUNCATE TABLE та відмінність від DELETE..... 16

3.8 Видалення таблиць (DROP TABLE)..... 18

3.9 Масове генерування тестових даних..... 20

Висновки..... 21

ДОДАТОК А. Скрипт створення бази даних і таблиць..... 23

ДОДАТОК Б. Скрипт наповнення таблиць..... 29

ДОДАТОК В. Скрипти ALTER, UPDATE/DELETE, TRUNCATE, DROP і генерування даних..... 34

ЛАБОРАТОРНА РОБОТА № 3. ЗНАЙОМСТВО З МОВОЮ SQL. СТВОРЕННЯ, РЕДАГУВАННЯ І ВИДАЛЕННЯ БАЗ ДАНИХ ТА ТАБЛИЦЬ

Мета: практичне освоєння мови SQL (Structured Query Language) як основного інструмента взаємодії з реляційною системою керування базами даних.

Вихідні дані. Фізичну модель бази даних STEM-студії «Стемік» (18 таблиць), спроєктовану в лабораторній роботі № 2 на основі поточної схеми робочого проєкту, реалізовано засобами мови SQL в окремій базі `stemik_lab3`. СУБД – PostgreSQL 16, та сама, що й у проєкті (Supabase). Скрипти виконувались у консольному клієнті `psql` у порядку їх нумерації; усі файли розміщено в каталозі `docs/db-labs/lab3/sql/` репозиторію (табл. 3.1).

Таблиця 3.1 – Скрипти лабораторної роботи

Файл	Завдання	Що робить
`00_create_database.sql`	1	CREATE DATABASE stemik_lab3
`01_create_tables.sql`	2	18 таблиць CREATE TABLE, 86 іменованих обмежень
`02_insert.sql`	4	тестові дані: 287 рядків у 18 таблицях
`03_alter.sql`	3	ALTER TABLE: стовпці, типи, обмеження
`04_update_delete.sql`	5	UPDATE і DELETE з умовами WHERE
`05_truncate.sql`	6	TRUNCATE TABLE проти DELETE
`06_drop.sql`	7	DROP TABLE і залежності FK
`07_generate.sql`	—	масове генерування тестових даних (розд. 10 методичних вказівок)

Скрипт `ALTER TABLE` виконується після наповнення таблиць: так видно, як зміна структури впливає на вже наявні рядки (наприклад, чому не можна просто додати стовпець `NOT NULL` до заповненої таблиці).

3.1 Підмови SQL, використані в роботі

Таблиця 3.2 – Оператори роботи за підмовами SQL

Підмова	Оператори	Де використано
DDL	CREATE DATABASE,	кроки 1, 2, 4, 6, 7

Підмова	Оператори	Де використано
	CREATE TABLE, ALTER TABLE, DROP TABLE, TRUNCATE TABLE, COMMENT	
DML	INSERT, UPDATE, DELETE (у т. ч. з RETURNING і FROM)	кроки 3, 5, 8
DQL	SELECT – перевірка перед змінами і контроль результату	усі кроки
TCL	BEGIN, ROLLBACK – безпечна демонстрація руйнівних операцій	кроки 5–8
DCL	GRANT, REVOKE – у цій роботі не використовувались (у проєкті – права ролей anon/authenticated у Supabase)	—

Особливість PostgreSQL, якою скористались у роботі: на відміну від багатьох СУБД, оператори DDL тут **транзакційні** – `DROP TABLE`, `TRUNCATE`, `ALTER TABLE` усередині `BEGIN … ROLLBACK` повністю скасовуються. Це дало змогу продемонструвати незворотні операції, не руйнуючи базу, потрібну для наступних лабораторних робіт.

3.2 Створення бази даних (CREATE DATABASE)

Базу створено з явним кодуванням UTF-8 (дані містять кирилицю) на основі шаблону `template0`, що дозволяє задати власні параметри локалі. Назва – коротка, англійською, без пробілів. У PostgreSQL немає оператора `USE`: до нової бази підключаються окремим з'єднанням (у `psql` – метакомандою `\c stemik_lab3`).

```
DROP DATABASE IF EXISTS stemik_lab3;

CREATE DATABASE stemik_lab3
    WITH ENCODING   = 'UTF8'
         TEMPLATE   = template0
         LC_COLLATE = 'C.UTF-8'
         LC_CTYPE   = 'C.UTF-8';

COMMENT ON DATABASE stemik_lab3 IS 'STEM-студія «Стемік»: облік занять, відвідувань,
оплат і 3D-друку';

-- У PostgreSQL немає оператора USE: до нової бази підключаються окремо
-- (у psql — метакомандою \c stemik_lab3).
```

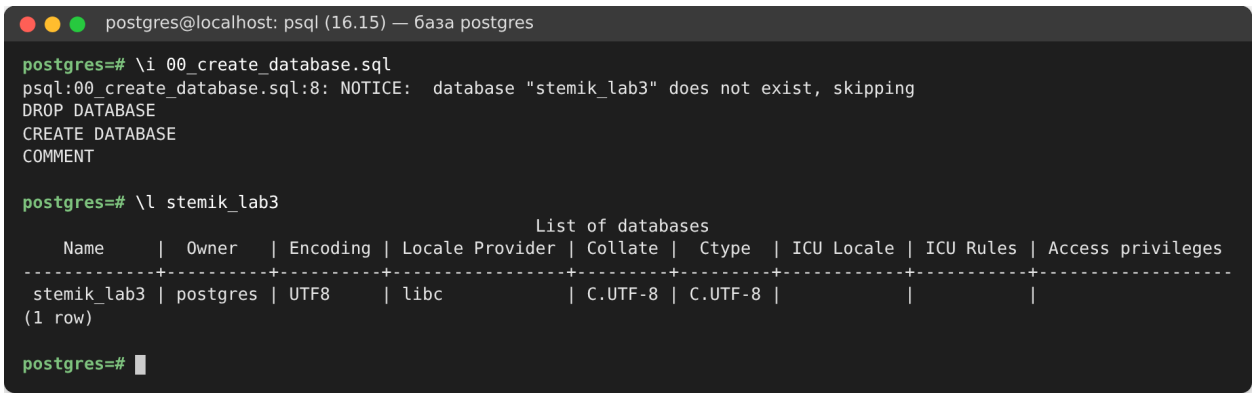


Рисунок 3.1 – Створення бази даних stemik_lab3 і її параметри (\l)

3.3 Створення таблиць (CREATE TABLE)

Порядок створення. Таблицю можна створити лише після всіх таблиць, на які посилаються її зовнішні ключі. Тому таблиці згруповано за рівнями: рівень таблиці дорівнює 1 + найбільший рівень таблиць, на які вона посилається (рис. 3.2). Незалежні довідники (рівень 0) створюються першими, асоціативна таблиця `lesson_student` – після обох сторін зв’язку M:N, а `attendance`, `work` і `work_cost`, що посилаються на неї складеним ключем, – останніми.

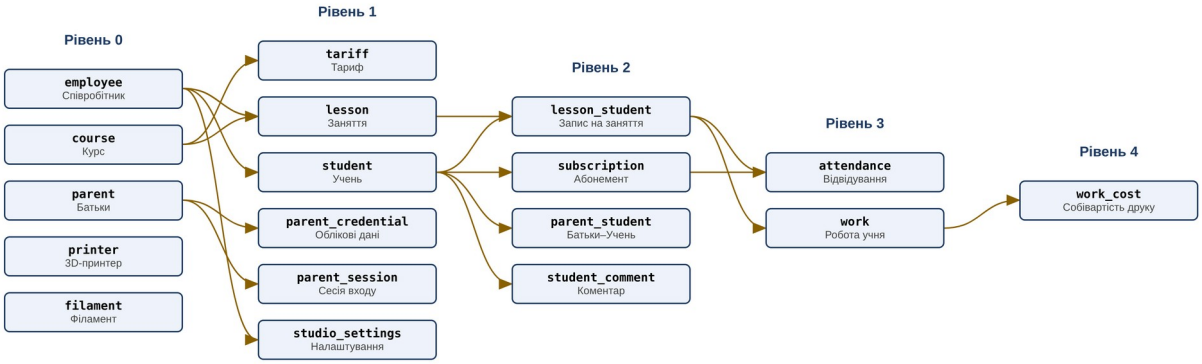


Рисунок 3.2 – Порядок створення таблиць за рівнями залежностей (стрілка – від таблиці, на яку посилаються, до тієї, що посилається; показано зв’язки, які визначають рівень)

Іменування обмежень. Усі 86 обмежень мають явні назви за єдиною схемою (табл. 3.3) – автоматично згенерованих назв у базі немає. Це спрощує подальші `ALTER TABLE … DROP CONSTRAINT` і робить повідомлення про

помилки зрозумілими: з тексту `violates foreign key constraint "fk_subscription_student"` одразу видно таблицю і зв’язок.

Таблиця 3.3 – Схема іменування обмежень

Префікс	Обмеження	Приклад	Кількість
`pk_`	PRIMARY KEY	`pk_attendance`	18
`fk_`	FOREIGN KEY	`fk_attendance_lesson_student`	25
`uq_`	UNIQUE	`uq_attendance_lesson_student`	8
`chk_`	CHECK	`chk_attendance_only_present`	35

Порівняно з DDL-скриптом лабораторної роботи № 2 обмеження оголошено на рівні таблиці (`CONSTRAINT <назва> …` після стовпців), а кілька однотипних перевірок однієї таблиці об’єднано в одне іменоване `CHECK` (наприклад, `chk_attendance_money` – сума, ціна друку і знижка). Типи, ключі, правила каскадних дій і бізнес-правила – ті самі. Фрагмент скрипту:

```
CREATE TABLE course (
    course_id          SMALLINT      GENERATED ALWAYS AS IDENTITY,
    name               VARCHAR(60)   NOT NULL,
    direction          VARCHAR(10)   NOT NULL,
    age_min            SMALLINT      NOT NULL,
    age_max            SMALLINT      NOT NULL,
    online_print_price NUMERIC(10,2),
    active             BOOLEAN       NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_course            PRIMARY KEY (course_id),
    CONSTRAINT uq_course_name       UNIQUE (name),
    CONSTRAINT chk_course_direction CHECK (direction IN ('3D', 'VR', 'ROBO')),
    CONSTRAINT chk_course_age       CHECK (age_min BETWEEN 4 AND 18
                                           AND age_max BETWEEN 4 AND 18
                                           AND age_min <= age_max),
    CONSTRAINT chk_course_print     CHECK (online_print_price >= 0)
);

CREATE TABLE attendance (
    attendance_id    UUID          NOT NULL DEFAULT gen_random_uuid(),
    lesson_id        UUID          NOT NULL,
    student_id       UUID          NOT NULL,
    status           VARCHAR(10)   NOT NULL,
    payment_method   VARCHAR(15),
    amount           NUMERIC(10,2) NOT NULL DEFAULT 0,
    discount_percent NUMERIC(5,2)  NOT NULL DEFAULT 0,
    subscription_id  UUID,
    print_enabled    BOOLEAN       NOT NULL DEFAULT FALSE,
    print_price      NUMERIC(10,2),
    marked_by        UUID,
    marked_at        TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CONSTRAINT pk_attendance                PRIMARY KEY (attendance_id),
    CONSTRAINT uq_attendance_lesson_student UNIQUE (lesson_id, student_id),
    CONSTRAINT fk_attendance_lesson_student FOREIGN KEY (lesson_id, student_id)
                                            REFERENCES lesson_student (lesson_id,
student_id)
                                            ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_attendance_subscription   FOREIGN KEY (subscription_id)
                                            REFERENCES subscription (subscription_id)
                                            ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_attendance_employee       FOREIGN KEY (marked_by) REFERENCES employee
(employee_id)
                                            ON DELETE SET NULL ON UPDATE CASCADE,
```

```
    CONSTRAINT chk_attendance_status        CHECK (status IN ('present', 'absent',
'excused')),
    CONSTRAINT chk_attendance_payment       CHECK (payment_method IN
                                                   ('card', 'cash', 'subscription',
'certificate')),
    CONSTRAINT chk_attendance_money         CHECK (amount >= 0 AND print_price >= 0
                                                   AND discount_percent BETWEEN 0 AND 100),
    CONSTRAINT chk_attendance_only_present  CHECK (status = 'present'
                                                   OR (payment_method IS NULL AND amount =
0)),
    CONSTRAINT chk_attendance_subscription  CHECK (subscription_id IS NULL
                                                   OR payment_method = 'subscription')
);
```

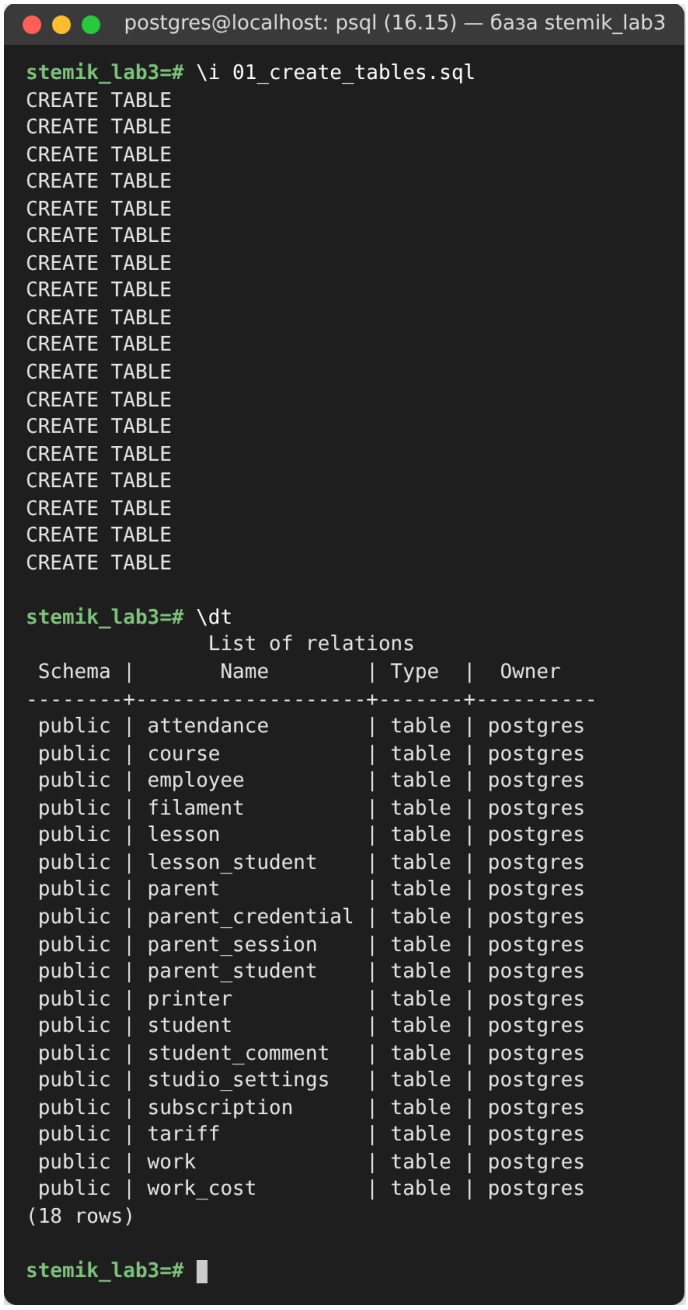


Рисунок 3.3 – Виконання скрипту створення таблиць і перелік 18 таблиць (\dt)

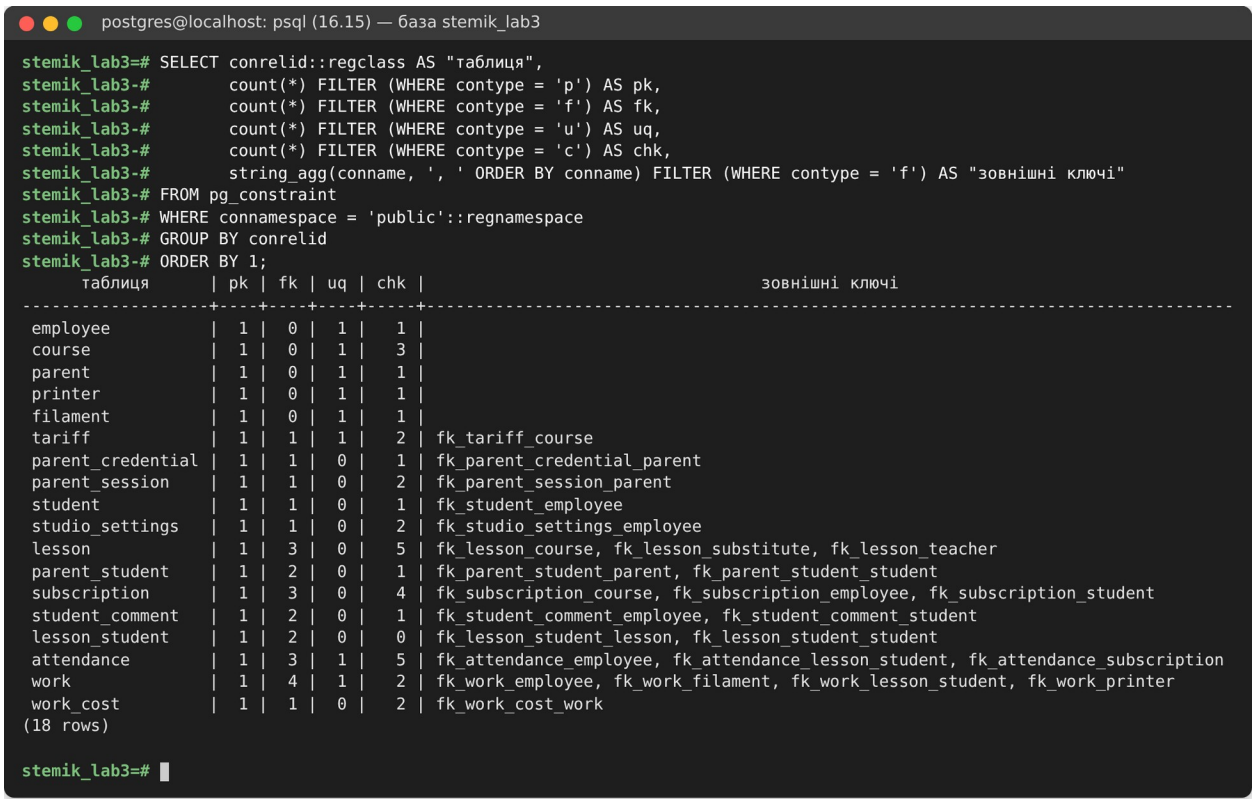


Рисунок 3.4 – Обмеження кожної таблиці за видами та назви зовнішніх ключів (з системного каталогу pg_constraint)

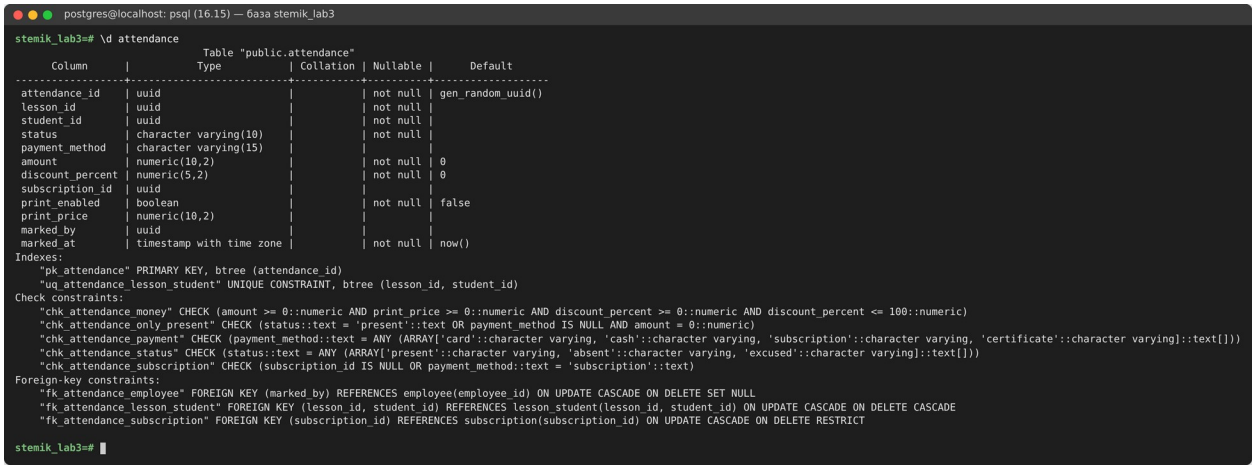


Рисунок 3.5 – Структура таблиці attendance: іменовані первинний, унікальний, зовнішні ключі та CHECK-обмеження

3.4 Наповнення таблиць даними (INSERT)

Кожну основну таблицю заповнено 10–15 і більше записами (табл. 3.4); довідники (`course`, `printer`, `filament`, `studio_settings`) містять стільки рядків, скільки потрібно предметній області: усі 9 курсів студії, 3 принтери, 6 видів пластику і єдиний рядок налаштувань. Дані узгоджені між собою:

зовнішні ключі посилаються на наявні рядки, ціни занять відповідають тарифам, абонементи – курсам, на які записані учні, а оплати – правилам відвідування. Імена й телефони вигадані.

Таблиця 3.4 – Кількість тестових записів

Таблиця	Рядків	Таблиця	Рядків
`employee`	10	`lesson`	15
`course`	9	`lesson_student`	48
`tariff`	54	`attendance`	44
`parent`	12	`subscription`	10
`parent_credential`	8	`work`	13
`parent_session`	6	`work_cost`	9
`student`	15	`student_comment`	10
`parent_student`	14	`printer` / `filament` / `studio_settings`	3 / 6 / 1

Використані прийоми вставки:

– **кілька рядків в одному INSERT** – співробітники, курси, батьки, принтери, пластик;

– **стовпці перелiковано явно**, а первинні ключі не вказуються: `IDENTITY` і `UUID DEFAULT gen_random_uuid()` генерує СУБД. Для `GENERATED ALWAYS AS IDENTITY` явне значення взагалі заборонене – СУБД поверне помилку `cannot insert a non-DEFAULT value into column`;

– **INSERT … SELECT з JOIN за природними ключами** – зовнішній ключ знаходиться за email викладача чи назвою курсу, а не вписується як UUID;

– **CROSS JOIN** курсів із переліком типів тарифу дав одразу 54 тарифи одним оператором;

– **unnest(string_to_array(…))** розгортає перелік імен учнів кожного заняття в рядки `lesson_student`;

– **залишок від ділення** номера рядка (`row_number() … % 4`) розподіляє способи оплати по колу: картка / готівка / картка / сертифікат; винятки (відсутні, ВПП, борг) задано окремим списком.

```
INSERT INTO employee (email, last_name, first_name, role, active) VALUES
    ('director@stemik.test',  'Коваленко', 'Ірина',    'superadmin', TRUE),
    ('admin@stemik.test',     'Мельник',   'Олег',     'admin',      TRUE),
    ('admin2@stemik.test',    'Литвин',    'Юлія',     'admin',      TRUE),
```

```
    ('shevchuk@stemik.test',  'Шевчук',    'Андрій',   'teacher',    TRUE),
    ('bondar@stemik.test',    'Бондар',    'Марія',    'teacher',    TRUE),
    ('tkachenko@stemik.test', 'Ткаченко',  'Дмитро',   'teacher',    TRUE),
    ('kravets@stemik.test',   'Кравець',   'Олена',    'teacher',    TRUE),
    ('polishchuk@stemik.test','Поліщук',   'Ігор',     'teacher',    TRUE),
    ('moroz@stemik.test',     'Мороз',     'Катерина', 'teacher',    TRUE),
    ('savchuk@stemik.test',   'Савчук',    'Віктор',   'teacher',    FALSE);

INSERT INTO tariff (course_id, tariff_type, teacher_pay, lesson_price, subscription_price)
SELECT c.course_id, t.tariff_type, t.pay, t.price,
       CASE WHEN t.sub IS NOT NULL THEN t.sub + (c.course_id % 3) * 40 END
FROM course AS c
CROSS JOIN (VALUES
    ('group_1',           250, 450, 1600),
    ('group_2_6',         350, 400, 1440),
    ('individual',        300, 600, 2200),
    ('individual_online', 250, 500, 1800),
    ('group_online',      300, 350, 1200),
    ('extra_service',      50, 150, NULL)
) AS t (tariff_type, pay, price, sub);
```

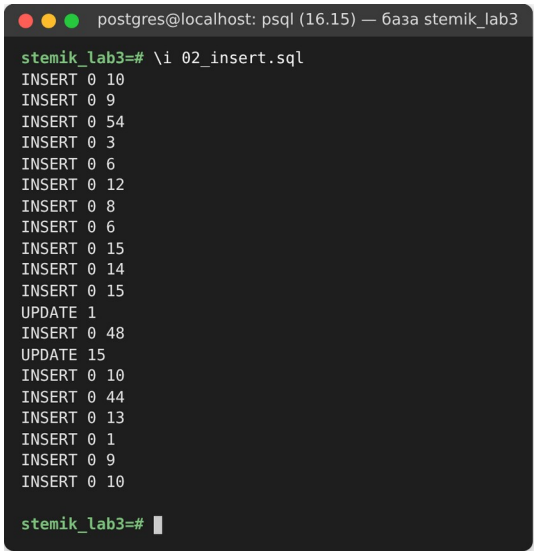


Рисунок 3.6 – Виконання скрипту наповнення: кількість вставлених і оновлених рядків



Рисунок 3.7 – Кількість рядків у кожній таблиці після наповнення

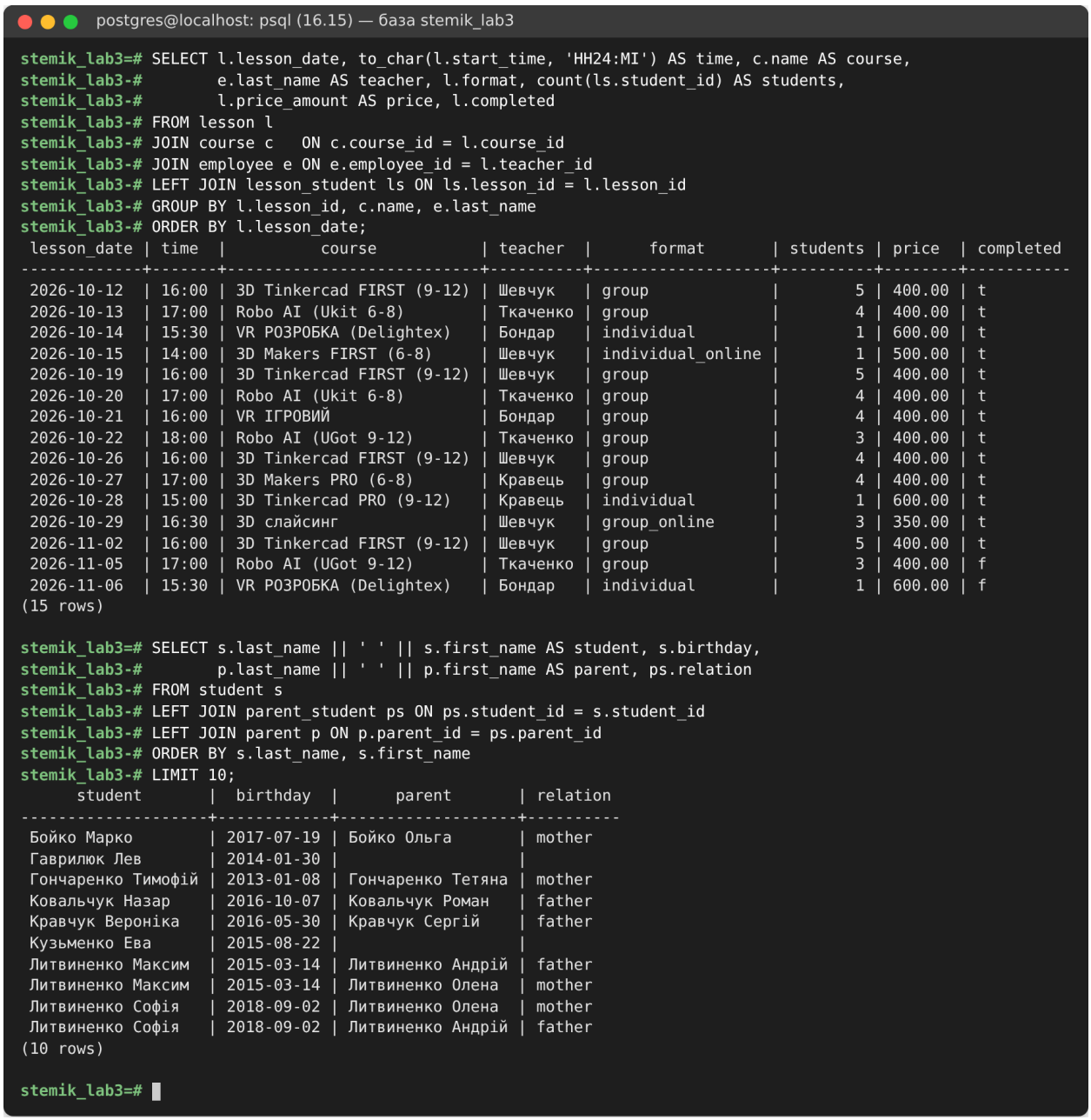


Рисунок 3.8 – Дані: заняття з викладачем, кількістю учнів і ціною; учні та їхні батьки

3.5 Зміна структури таблиць (ALTER TABLE)

Продемонстровано всі три сценарії з завдання – додавання стовпця, зміну типу, додавання й видалення обмеження – а також випадки, коли СУБД відмовляє, захищаючи наявні дані (табл. 3.5, рис. 3.9).

Таблиця 3.5 – Операції ALTER TABLE та їх результат

№	Операція	Результат
1	ADD COLUMN student.school VARCHAR(100)	для 15 наявних рядків значення NULL
2	ADD COLUMN	помилка: стовпець містив би

№	Операція	Результат
	employee.hire_date DATE NOT NULL	NULL
3	те саме з DEFAULT DATE '2025-09-01'	усі рядки отримали значення за замовчуванням
4	ALTER COLUMN student.phone TYPE VARCHAR(30)	розширення – безпечне
5	ALTER COLUMN course.name TYPE VARCHAR(10)	помилка: value too long – дані не обрізаються
6	lesson.duration_minutes TYPE INTEGER USING …	зміна числового типу з явним перетворенням
7	ADD CONSTRAINT chk_student_phone_format	формат +380XXXXXXXXX
8	UPDATE з телефоном «067 123 45 67»	відхилено новим обмеженням
9	ADD CONSTRAINT chk_student_has_notes	помилка: наявні рядки порушують умову
10	DROP CONSTRAINT chk_student_phone_format	видалено за назвою
11	RENAME COLUMN, DROP COLUMN	student повернуто до моделі

Після скрипту в базі свідомо залишено три зміни: `employee.hire_date` (дата прийому на роботу – знадобиться для запитів ЛР № 4), `student.phone VARCHAR(30)` і `lesson.duration_minutes INTEGER`. У PostgreSQL зміна типу записується як `ALTER COLUMN … TYPE`, а не `ALTER COLUMN` без ключового слова, як у MS SQL Server.

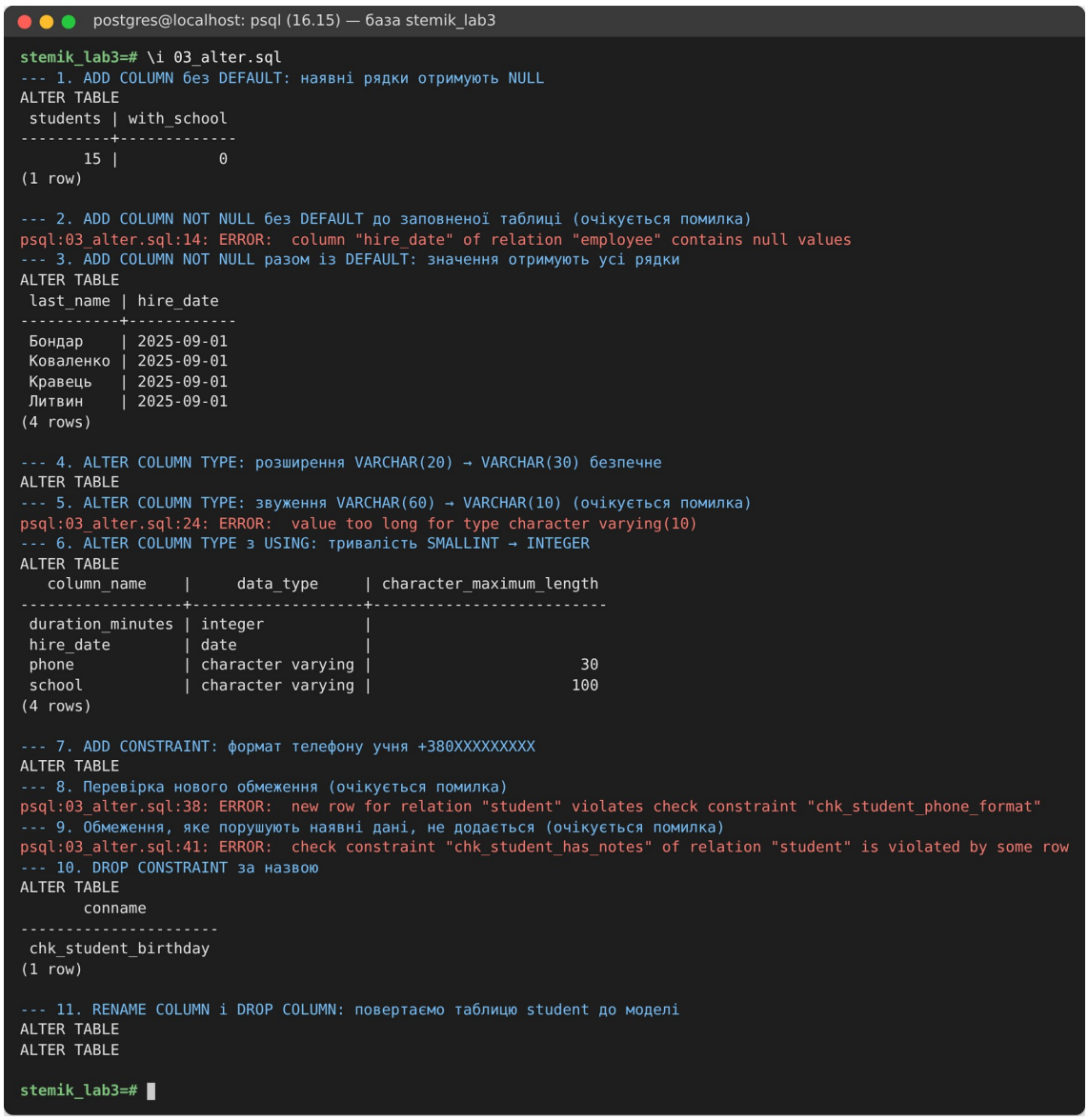


Рисунок 3.9 – Виконання ALTER TABLE: додавання стовпців, зміна типів, додавання і видалення обмежень

3.6 Оновлення і видалення записів (UPDATE, DELETE)

Кожна зміна виконується з умовою `WHERE`; перед масовою зміною спершу виконано `SELECT` з тією самою умовою, щоб побачити, які рядки буде зачеплено. Оператор `RETURNING` (розширення PostgreSQL) одразу показує змінені рядки. Сценарії взято з реальних процесів студії:

– **UPDATE … FROM** перераховує залишок усіх 10 абонементів за фактичними відвідуваннями: абонемент Максима використано повністю, його статус став `completed`;

– **UPDATE з умовою** підвищує на 10 % ціну групових занять лише 3D-курсів (5 рядків із 54);

– **UPDATE** закриває борг: заняття, позначене «присутній» без оплати, оплачено готівкою;

– **DELETE з умовою** видаляє неактуальний коментар про борг і завершує сесію кабінету батьків, неактивну з 31.10;

– **DELETE учня з історією** відхилено: на нього посилається абонемент (`fk_subscription_student`, `ON DELETE RESTRICT`);

– **DELETE з каскадом**: видалення кабінету батьків автоматично видаляє його зв’язок із дитиною (12 → 11 кабінетів, 14 → 13 зв’язків; у транзакції з відкатом);

– **UPDATE без WHERE** змінив кабінет у всіх 15 заняттях – наочна ілюстрація небезпеки; зміну відкочено `ROLLBACK`.

```
UPDATE subscription AS sb
SET remaining_lessons = sb.total_lessons - u.used,
    status = CASE WHEN sb.total_lessons - u.used = 0 THEN 'completed' ELSE 'active' END
FROM (SELECT subscription_id, count(*) AS used
      FROM attendance
      WHERE subscription_id IS NOT NULL
      GROUP BY subscription_id) AS u
WHERE u.subscription_id = sb.subscription_id;
```

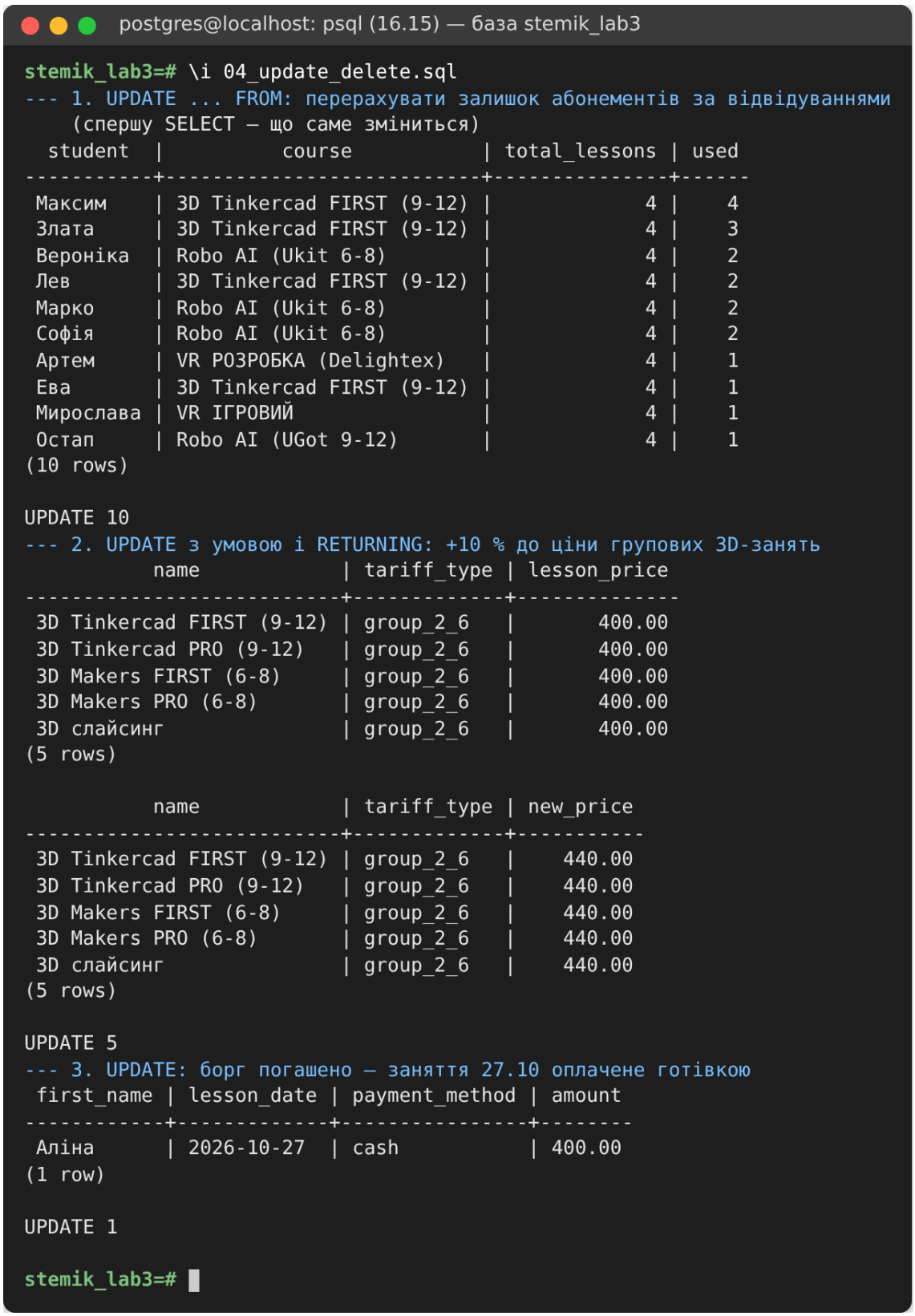


Рисунок 3.10 – UPDATE: перерахунок залишку абонементів, підвищення тарифів, закриття боргу

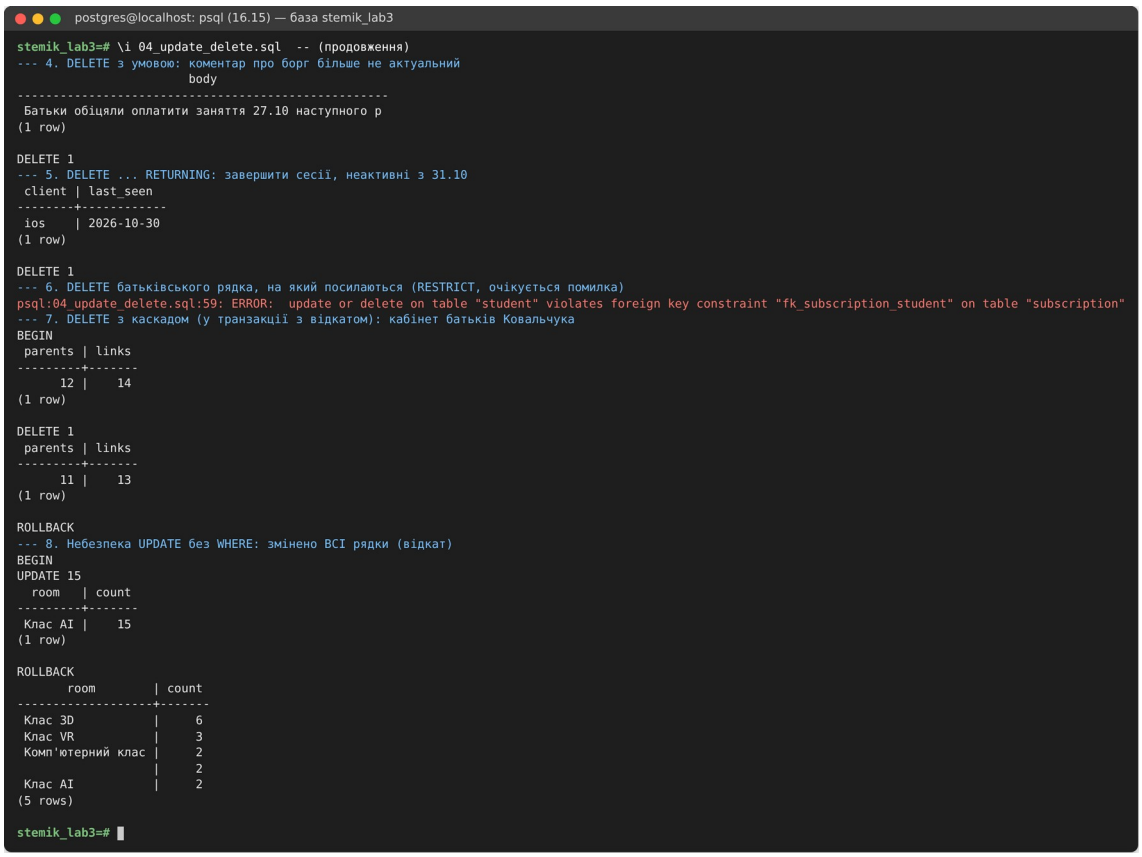


Рисунок 3.11 – DELETE з умовою, відмова через RESTRICT, каскадне видалення і небезпека UPDATE без WHERE

3.7 TRUNCATE TABLE та відмінність від DELETE

Обидві команди можуть повністю очистити таблицю, але працюють принципово по-різному. Відмінності, перевірені на базі `stemik_lab3`, наведено в табл. 3.6 і на рис. 3.12–3.13.

Таблиця 3.6 – Порівняння DELETE і TRUNCATE TABLE (PostgreSQL 16)

Властивість	DELETE FROM t	TRUNCATE TABLE t
Підмова	DML, видаляє рядки поодинці	DDL, звільняє сторінки таблиці цілком
Умова WHERE	так	ні – лише вся таблиця
Лічильник IDENTITY	продовжує (новий id 55)	продовжує за замовчуванням (CONTINUE IDENTITY); з RESTART IDENTITY – з 1
Таблиця, на яку посилається FK	можна: FK перевіряється для кожного рядка (printer → SET NULL у work)	помилка; TRUNCATE … CASCADE очищає ще й усі залежні таблиці (work, work_cost)
Тригери ON DELETE	спрацьовують	не спрацьовують
Швидкість, 500 000 рядків	≈ 66 мс	≈ 7 мс
Відкат у транзакції	так	так – у PostgreSQL TRUNCATE транзакційний

Окремо видно, що значення послідовності `IDENTITY` не повертається при `ROLLBACK`: після відкоту першого тесту наступна вставка отримала `tariff_id` 56, а не 55. Послідовності в PostgreSQL навмисно нетранзакційні, тож «дірки» в нумерації сурогатних ключів – нормальне явище. Висновок: `TRUNCATE` – для швидкого повного очищення (наприклад, тестової таблиці перед повторним заповненням), `DELETE … WHERE` – для вибіркового видалення з перевіркою зв’язків.

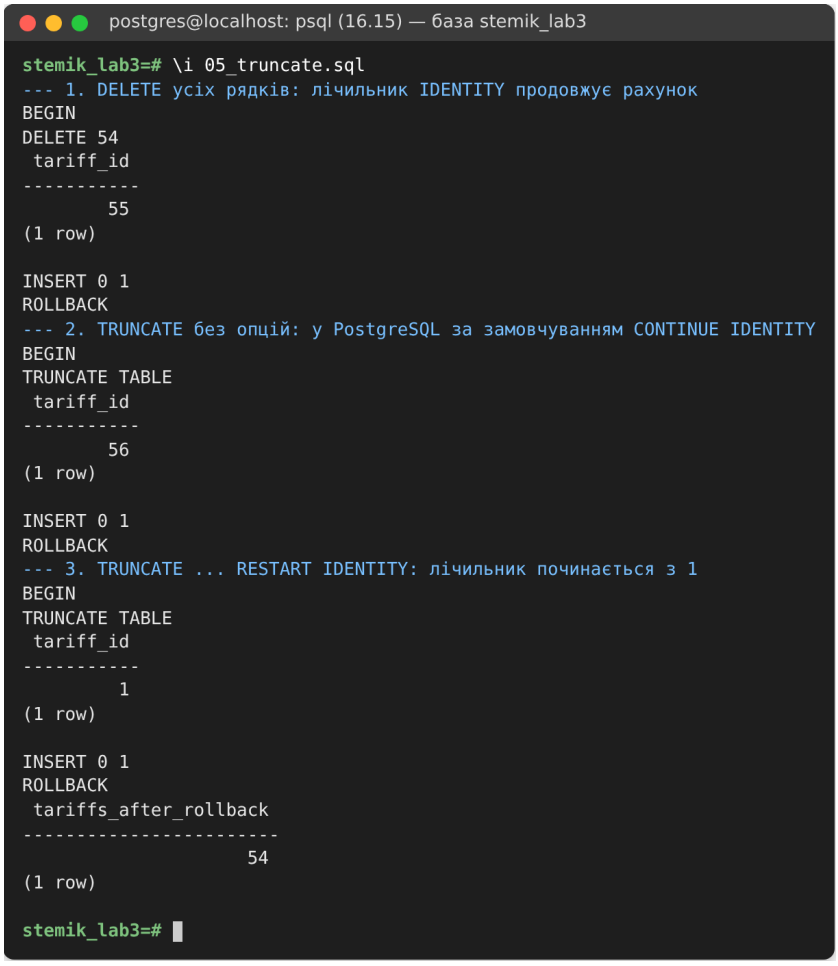


Рисунок 3.12 – DELETE і TRUNCATE: поведінка лічильника IDENTITY

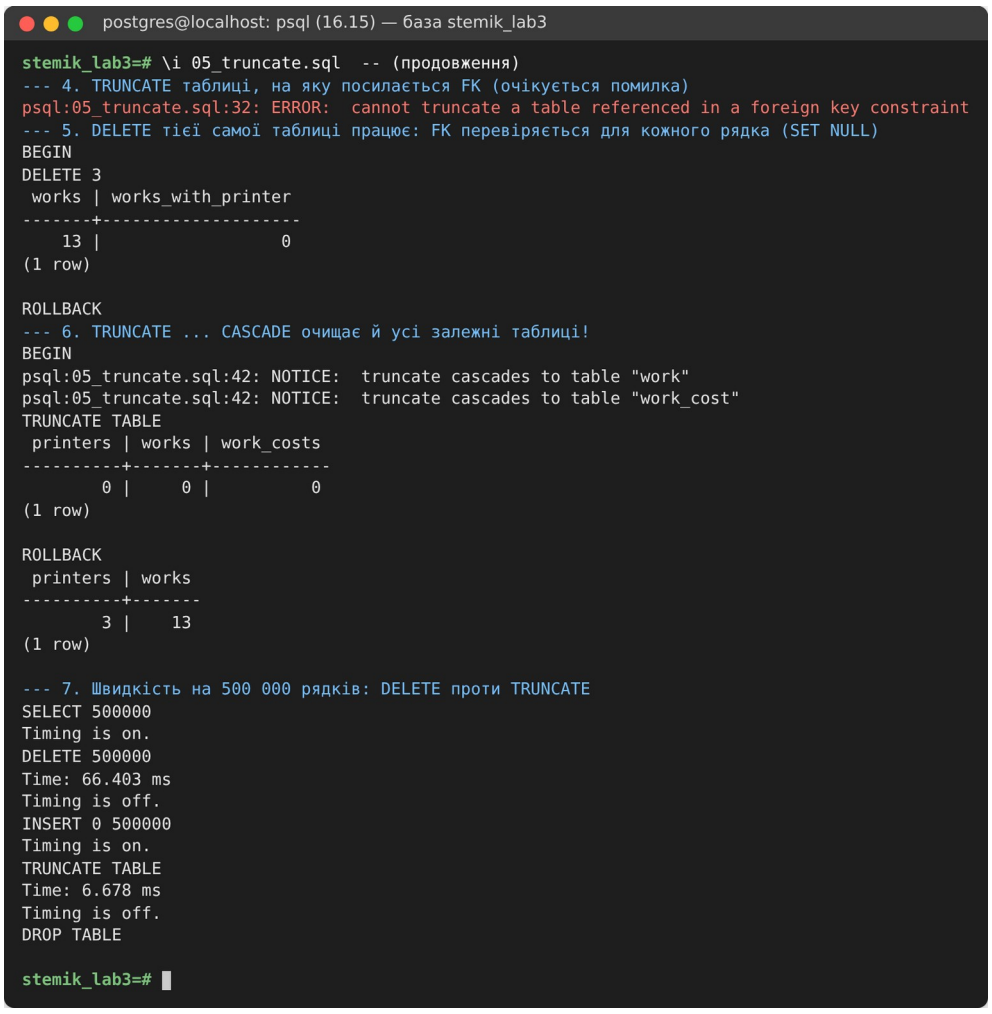


Рисунок 3.13 – TRUNCATE таблиці з зовнішніми ключами, TRUNCATE … CASCADE і порівняння швидкості

3.8 Видалення таблиць (DROP TABLE)

Спроба видалити батьківську таблицю `course` завершилась помилкою: на неї посилаються три зовнішні ключі – `fk_tariff_course`, `fk_subscription_course`, `fk_lesson_course` (рис. 3.14). СУБД перелічує залежні об’єкти й пропонує `DROP … CASCADE`. Правильний порядок видалення – зворотний до порядку створення (рис. 3.2): спершу `work_cost`, потім `attendance` і `work`, `lesson_student`, і лише наприкінці `course`. Варто розуміти, що `DROP TABLE course CASCADE` видаляє **не дочірні таблиці, а їхні зовнішні ключі**: таблиці `lesson`, `tariff`, `subscription` залишаються з усіма даними, але вже без зв’язку з курсом – тобто з’являються «осиротілі» значення `course_id`. «Листову» таблицю `student_comment`, на яку ніщо не

посилається, видалено без помилок, а `DROP TABLE IF EXISTS` для вже неіснуючої таблиці лише видав попередження. Усі сценарії виконано в транзакціях із відкатом: після скрипту в базі знову 18 таблиць.

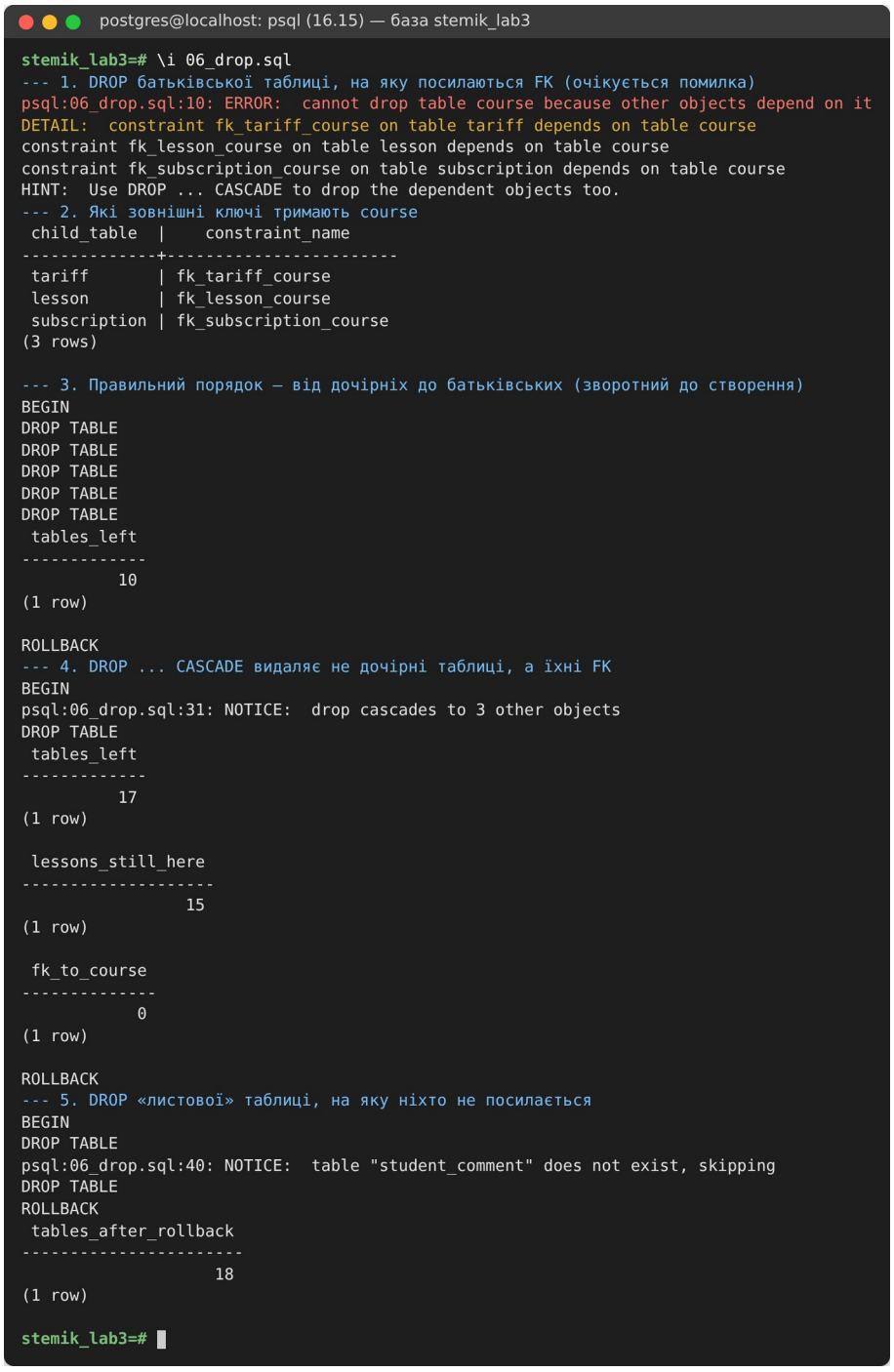


Рисунок 3.14 – DROP TABLE: помилка залежності, видалення у зворотному порядку, DROP … CASCADE

3.9 Масове генерування тестових даних

Прийоми з розділу 10 методичних вказівок (MS SQL Server) реалізовано засобами PostgreSQL (табл. 3.7). Усе виконано в транзакції з відкатом, тому основні дані не змінились.

Таблиця 3.7 – Прийоми генерування даних: MS SQL Server → PostgreSQL

MS SQL Server	PostgreSQL	Що згенеровано
WHILE з DECLARE @i	WHILE … LOOP у блоці DO	100 тестових учнів
CROSS JOIN sys.objects	generate_series(1, 1000)	1000 кабінетів батьків одним INSERT
WITH … OPTION (MAXRECURSION)	WITH RECURSIVE (обмеження глибини немає)	30 занять щотижневої серії до 31.05.2027
ORDER BY NEWID(), TOP 1	ORDER BY random() LIMIT 1	випадковий активний викладач заняття
seq % n	seq % n, масиви ARRAY[…][i]	курси, кабінети й час занять по колу

Серія з 30 щотижневих занять відтворює реальну можливість застосунку – заняття з однаковим `series_id`, які можна редагувати разом. Порівняння швидкості підтвердило рекомендацію методичних вказівок: 20 000 рядків циклом `WHILE` вставляються ≈ 28 мс, а одним `INSERT … SELECT` із `generate_series` – ≈ 6 мс, тобто майже вп’ятеро швидше, бо замість 20 000 окремих операторів виконується одна масова вставка.

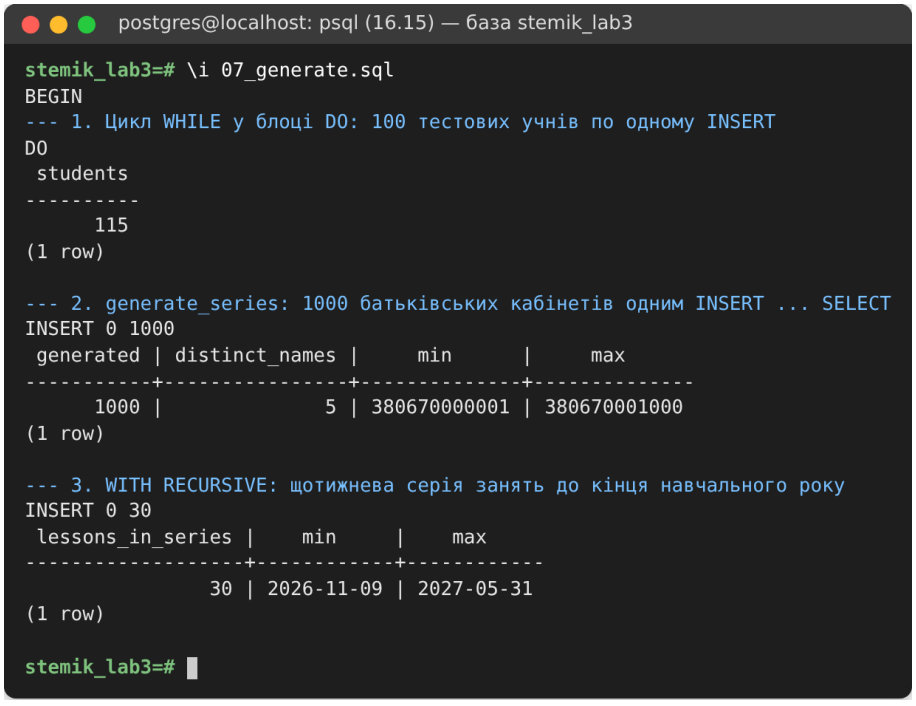


Рисунок 3.15 – Генерування: цикл WHILE, generate_series, рекурсивний CTE

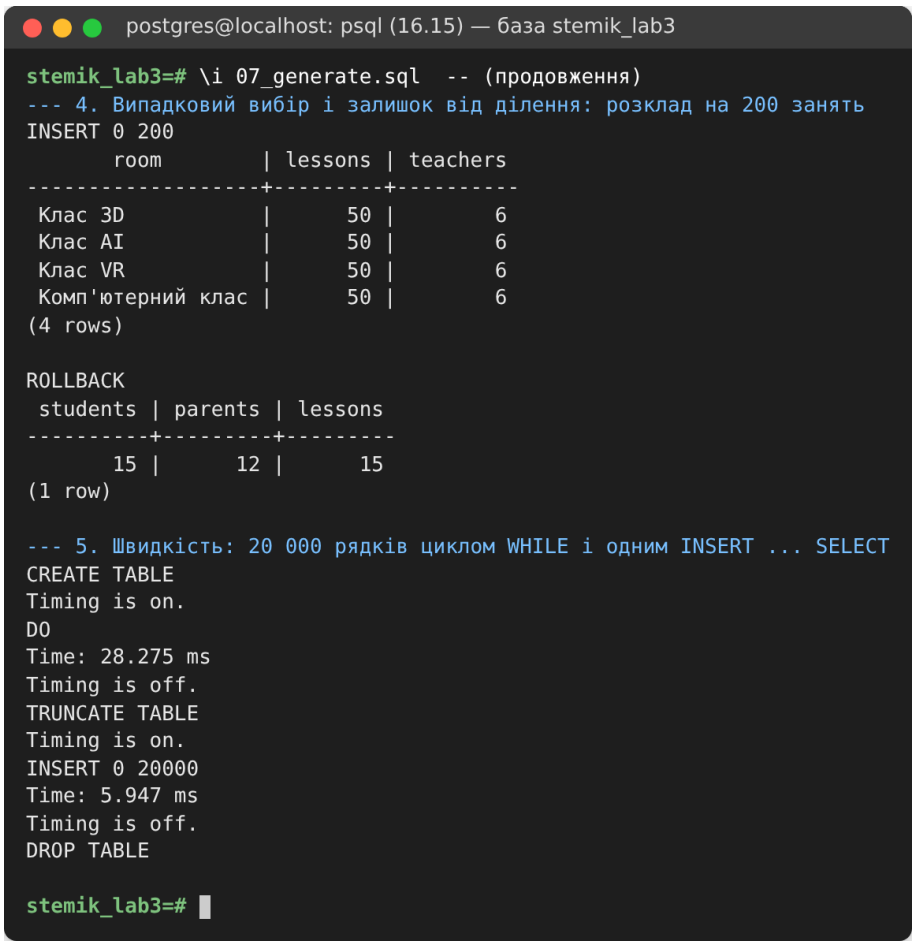


Рисунок 3.16 – Випадковий вибір і залишок від ділення; порівняння швидкості WHILE і generate_series

Висновки

У лабораторній роботі засобами мови SQL реалізовано базу даних STEM-студії «Стемік»: створено базу `stemik_lab3` і 18 таблиць у порядку залежностей з 86 явно іменованими обмеженнями (18 PK, 25 FK, 8 UNIQUE, 35 CHECK). Таблиці заповнено 287 узгодженими тестовими записами з використанням багаторядкових `INSERT`, `INSERT … SELECT`, `CROSS JOIN` і розподілу значень за залишком від ділення. Продемонстровано `ALTER TABLE` (додавання стовпців, зміну типів, додавання й видалення обмежень) і випадки, коли СУБД захищає наявні дані; `UPDATE` і `DELETE` з умовами та наслідки `RESTRICT` і `CASCADE`; відмінності `TRUNCATE` від `DELETE` (лічильник `IDENTITY`, зовнішні ключі, швидкість ≈ 7 мс проти ≈ 66 мс на 500 тис. рядків); видалення таблиць з урахуванням залежностей. Транзакційність

DDL у PostgreSQL дала змогу показати всі руйнівні операції без втрати даних, і база готова для запитів у лабораторній роботі № 4.

ДОДАТОК А. Скрипт створення бази даних і таблиць

Файли `docs/db-labs/lab3/sql/00_create_database.sql` і `01_create_tables.sql`.

```
-- ================================================================
-- Лабораторна робота №3. Крок 1: створення бази даних
-- Виконується з будь-якої іншої бази (наприклад, postgres):
--   psql -d postgres -f 00_create_database.sql
-- ================================================================

-- Повторний запуск: спершу прибрати базу, якщо вона вже є
DROP DATABASE IF EXISTS stemik_lab3;

CREATE DATABASE stemik_lab3
    WITH ENCODING   = 'UTF8'
         TEMPLATE   = template0
         LC_COLLATE = 'C.UTF-8'
         LC_CTYPE   = 'C.UTF-8';

COMMENT ON DATABASE stemik_lab3 IS 'STEM-студія «Стемік»: облік занять, відвідувань, оплат і 3D-друку';

-- У PostgreSQL немає оператора USE: до нової бази підключаються окремо
-- (у psql — метакомандою \c stemik_lab3).

-- ================================================================
-- Лабораторна робота №3. Крок 2: створення таблиць (CREATE TABLE)
-- База: stemik_lab3 (PostgreSQL 16). Модель — з лабораторної роботи №2.
--
-- Усі обмеження названо явно, щоб їх можна було змінити чи видалити
-- через ALTER TABLE ... DROP CONSTRAINT <назва>:
--   pk_<таблиця>             первинний ключ
--   fk_<таблиця>_<посилання> зовнішній ключ
--   uq_<таблиця>_<стовпці>   унікальність
--   chk_<таблиця>_<правило>  перевірка CHECK
--
-- Порядок створення — за рівнями залежностей:
--   рівень 0: employee, course, parent, printer, filament
--   рівень 1: tariff, parent_credential, parent_session, student,
--             studio_settings, lesson
--   рівень 2: parent_student, subscription, student_comment, lesson_student
--   рівень 3: attendance, work
--   рівень 4: work_cost
-- Рівень таблиці = 1 + найбільший рівень таблиць, на які вона посилається.
-- ================================================================

-- ----------------------------------------------------------------
-- Рівень 0: незалежні таблиці (без зовнішніх ключів)
-- ----------------------------------------------------------------
CREATE TABLE employee (
    employee_id  UUID         NOT NULL DEFAULT gen_random_uuid(),
    email        VARCHAR(255) NOT NULL,
    last_name    VARCHAR(50)  NOT NULL,
    first_name   VARCHAR(50)  NOT NULL,
    role         VARCHAR(20)  NOT NULL DEFAULT 'teacher',
    active       BOOLEAN      NOT NULL DEFAULT TRUE,
    photo_url    TEXT,
    created_at   TIMESTAMPTZ  NOT NULL DEFAULT now(),
    CONSTRAINT pk_employee       PRIMARY KEY (employee_id),
    CONSTRAINT uq_employee_email UNIQUE (email),
    CONSTRAINT chk_employee_role CHECK (role IN ('teacher', 'admin', 'superadmin'))
);

CREATE TABLE course (
    course_id          SMALLINT      GENERATED ALWAYS AS IDENTITY,
    name               VARCHAR(60)   NOT NULL,
    direction          VARCHAR(10)   NOT NULL,
    age_min            SMALLINT      NOT NULL,
    age_max            SMALLINT      NOT NULL,
    online_print_price NUMERIC(10,2),
    active             BOOLEAN       NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_course            PRIMARY KEY (course_id),
    CONSTRAINT uq_course_name       UNIQUE (name),
    CONSTRAINT chk_course_direction CHECK (direction IN ('3D', 'VR', 'ROBO')),
    CONSTRAINT chk_course_age       CHECK (age_min BETWEEN 4 AND 18
```

```
                                          AND age_max BETWEEN 4 AND 18
                                          AND age_min <= age_max),
    CONSTRAINT chk_course_print     CHECK (online_print_price >= 0)
);

CREATE TABLE parent (
    parent_id      UUID        NOT NULL DEFAULT gen_random_uuid(),
    phone          VARCHAR(15) NOT NULL,
    last_name      VARCHAR(50),
    first_name     VARCHAR(50),
    active         BOOLEAN     NOT NULL DEFAULT TRUE,
    last_login_at  TIMESTAMPTZ,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT pk_parent        PRIMARY KEY (parent_id),
    CONSTRAINT uq_parent_phone  UNIQUE (phone),
    CONSTRAINT chk_parent_phone CHECK (phone ~ '^[0-9]{10,15}$')
);

CREATE TABLE printer (
    printer_id            SMALLINT     GENERATED ALWAYS AS IDENTITY,
    name                  VARCHAR(60)  NOT NULL,
    power_watts           NUMERIC(6,1) NOT NULL DEFAULT 0,
    depreciation_per_hour NUMERIC(8,2) NOT NULL DEFAULT 0,
    active                BOOLEAN      NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_printer         PRIMARY KEY (printer_id),
    CONSTRAINT uq_printer_name    UNIQUE (name),
    CONSTRAINT chk_printer_values CHECK (power_watts >= 0 AND depreciation_per_hour >= 0)
);

CREATE TABLE filament (
    filament_id  SMALLINT     GENERATED ALWAYS AS IDENTITY,
    material     VARCHAR(20)  NOT NULL,
    color        VARCHAR(30)  NOT NULL DEFAULT '',
    price_per_kg NUMERIC(8,2) NOT NULL DEFAULT 0,
    active       BOOLEAN      NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_filament                PRIMARY KEY (filament_id),
    CONSTRAINT uq_filament_material_color UNIQUE (material, color),
    CONSTRAINT chk_filament_price         CHECK (price_per_kg >= 0)
);

-- ---------------------------------------------------------------------
-- Рівень 1: посилаються лише на таблиці рівня 0
-- ---------------------------------------------------------------------
CREATE TABLE tariff (
    tariff_id          INT           GENERATED ALWAYS AS IDENTITY,
    course_id          SMALLINT      NOT NULL,
    tariff_type        VARCHAR(20)   NOT NULL,
    teacher_pay        NUMERIC(10,2) NOT NULL DEFAULT 0,
    lesson_price       NUMERIC(10,2) NOT NULL DEFAULT 0,
    subscription_price NUMERIC(10,2),
    CONSTRAINT pk_tariff             PRIMARY KEY (tariff_id),
    CONSTRAINT fk_tariff_course      FOREIGN KEY (course_id) REFERENCES course (course_id)
                                     ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT uq_tariff_course_type UNIQUE (course_id, tariff_type),
    CONSTRAINT chk_tariff_type       CHECK (tariff_type IN ('group_1', 'group_2_6', 'individual',
                                                            'individual_online', 'group_online',
                                                            'extra_service')),
    CONSTRAINT chk_tariff_money      CHECK (teacher_pay >= 0 AND lesson_price >= 0
                                            AND subscription_price >= 0)
);

CREATE TABLE parent_credential (
    parent_id       UUID        NOT NULL,
    password_hash   TEXT        NOT NULL,
    password_set_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    failed_attempts SMALLINT    NOT NULL DEFAULT 0,
    locked_until    TIMESTAMPTZ,
    CONSTRAINT pk_parent_credential        PRIMARY KEY (parent_id),
    CONSTRAINT fk_parent_credential_parent FOREIGN KEY (parent_id) REFERENCES parent (parent_id)
                                           ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT chk_parent_credential_tries CHECK (failed_attempts >= 0)
);

CREATE TABLE parent_session (
    token_hash    CHAR(64)    NOT NULL,
    parent_id     UUID        NOT NULL,
    client        VARCHAR(20),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_seen_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at    TIMESTAMPTZ NOT NULL,
```

```
    CONSTRAINT pk_parent_session         PRIMARY KEY (token_hash),
    CONSTRAINT fk_parent_session_parent  FOREIGN KEY (parent_id) REFERENCES parent (parent_id)
                                         ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT chk_parent_session_client CHECK (client IN ('web', 'android', 'ios', 'macos')),
    CONSTRAINT chk_parent_session_expiry CHECK (expires_at > created_at)
);

CREATE TABLE student (
    student_id  UUID        NOT NULL DEFAULT gen_random_uuid(),
    last_name   VARCHAR(50) NOT NULL,
    first_name  VARCHAR(50) NOT NULL,
    birthday    DATE,
    phone       VARCHAR(20),
    notes       TEXT,
    photo_url   TEXT,
    active      BOOLEAN     NOT NULL DEFAULT TRUE,
    created_by  UUID,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT pk_student          PRIMARY KEY (student_id),
    CONSTRAINT fk_student_employee FOREIGN KEY (created_by) REFERENCES employee (employee_id)
                                   ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT chk_student_birthday CHECK (birthday BETWEEN DATE '2005-01-01' AND CURRENT_DATE)
);

CREATE TABLE studio_settings (
    id                     BOOLEAN      NOT NULL DEFAULT TRUE,
    calc_start_date        DATE,
    electricity_tariff     NUMERIC(6,2) NOT NULL DEFAULT 0,
    filament_waste_percent NUMERIC(5,2) NOT NULL DEFAULT 0,
    updated_by             UUID,
    updated_at             TIMESTAMPTZ  NOT NULL DEFAULT now(),
    CONSTRAINT pk_studio_settings          PRIMARY KEY (id),
    CONSTRAINT fk_studio_settings_employee FOREIGN KEY (updated_by) REFERENCES employee
(employee_id)
                                           ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT chk_studio_settings_single  CHECK (id),
    CONSTRAINT chk_studio_settings_values  CHECK (electricity_tariff >= 0
                                                  AND filament_waste_percent BETWEEN 0 AND 100)
);

CREATE TABLE lesson (
    lesson_id        UUID          NOT NULL DEFAULT gen_random_uuid(),
    course_id        SMALLINT      NOT NULL,
    teacher_id       UUID          NOT NULL,
    substitute_id    UUID,
    series_id        UUID,
    lesson_date      DATE          NOT NULL,
    start_time       TIME,
    duration_minutes SMALLINT      NOT NULL DEFAULT 60,
    format           VARCHAR(20)   NOT NULL DEFAULT 'group',
    room             VARCHAR(30),
    theme            VARCHAR(200)  NOT NULL DEFAULT '',
    extra_service    BOOLEAN       NOT NULL DEFAULT FALSE,
    print_default    BOOLEAN       NOT NULL DEFAULT FALSE,
    completed        BOOLEAN       NOT NULL DEFAULT FALSE,
    payroll_amount   NUMERIC(10,2) NOT NULL DEFAULT 0,
    price_amount     NUMERIC(10,2) NOT NULL DEFAULT 0,
    CONSTRAINT pk_lesson             PRIMARY KEY (lesson_id),
    CONSTRAINT fk_lesson_course      FOREIGN KEY (course_id) REFERENCES course (course_id)
                                     ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_lesson_teacher     FOREIGN KEY (teacher_id) REFERENCES employee (employee_id)
                                     ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_lesson_substitute  FOREIGN KEY (substitute_id) REFERENCES employee
(employee_id)
                                     ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT chk_lesson_duration   CHECK (duration_minutes BETWEEN 15 AND 240),
    CONSTRAINT chk_lesson_format     CHECK (format IN ('group', 'individual',
                                                       'individual_online', 'group_online')),
    CONSTRAINT chk_lesson_room       CHECK (room IN ('Клас AI', 'Клас VR',
                                                     'Комп''ютерний клас', 'Клас 3D')),
    CONSTRAINT chk_lesson_money      CHECK (payroll_amount >= 0 AND price_amount >= 0),
    CONSTRAINT chk_lesson_substitute CHECK (substitute_id <> teacher_id)
);

-- ---------------------------------------------------------------------
-- Рівень 2
-- ---------------------------------------------------------------------
CREATE TABLE parent_student (
    parent_id   UUID        NOT NULL,
    student_id  UUID        NOT NULL,
```

```
    relation    VARCHAR(20) NOT NULL DEFAULT 'parent',
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT pk_parent_student          PRIMARY KEY (parent_id, student_id),
    CONSTRAINT fk_parent_student_parent   FOREIGN KEY (parent_id) REFERENCES parent (parent_id)
                                          ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_parent_student_student  FOREIGN KEY (student_id) REFERENCES student
(student_id)
                                          ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT chk_parent_student_relation CHECK (relation IN ('mother', 'father', 'parent',
'guardian'))
);

CREATE TABLE subscription (
    subscription_id   UUID          NOT NULL DEFAULT gen_random_uuid(),
    student_id        UUID          NOT NULL,
    course_id         SMALLINT      NOT NULL,
    total_lessons     SMALLINT      NOT NULL DEFAULT 4,
    remaining_lessons SMALLINT      NOT NULL DEFAULT 4,
    total_price       NUMERIC(10,2) NOT NULL DEFAULT 0,
    payment_method    VARCHAR(10),
    status            VARCHAR(10)   NOT NULL DEFAULT 'active',
    purchased_at      DATE          NOT NULL DEFAULT CURRENT_DATE,
    purchased_by      UUID,
    CONSTRAINT pk_subscription           PRIMARY KEY (subscription_id),
    CONSTRAINT fk_subscription_student   FOREIGN KEY (student_id) REFERENCES student (student_id)
                                         ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_subscription_course    FOREIGN KEY (course_id) REFERENCES course (course_id)
                                         ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_subscription_employee  FOREIGN KEY (purchased_by) REFERENCES employee
(employee_id)
                                         ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT chk_subscription_lessons  CHECK (total_lessons > 0
                                                AND remaining_lessons BETWEEN 0 AND
total_lessons),
    CONSTRAINT chk_subscription_price    CHECK (total_price >= 0),
    CONSTRAINT chk_subscription_payment  CHECK (payment_method IN ('card', 'cash')),
    CONSTRAINT chk_subscription_status   CHECK (status IN ('active', 'completed', 'cancelled'))
);

CREATE TABLE student_comment (
    comment_id  UUID        NOT NULL DEFAULT gen_random_uuid(),
    student_id  UUID        NOT NULL,
    author_id   UUID,
    body        TEXT        NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT pk_student_comment          PRIMARY KEY (comment_id),
    CONSTRAINT fk_student_comment_student  FOREIGN KEY (student_id) REFERENCES student
(student_id)
                                           ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_student_comment_employee FOREIGN KEY (author_id) REFERENCES employee
(employee_id)
                                           ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT chk_student_comment_body    CHECK (length(trim(body)) > 0)
);

-- ---------------------------------------------------------------------
-- Рівень 2 (продовження): асоціативна таблиця M:N заняття ↔ учень
-- ---------------------------------------------------------------------
CREATE TABLE lesson_student (
    lesson_id   UUID NOT NULL,
    student_id  UUID NOT NULL,
    CONSTRAINT pk_lesson_student         PRIMARY KEY (lesson_id, student_id),
    CONSTRAINT fk_lesson_student_lesson  FOREIGN KEY (lesson_id) REFERENCES lesson (lesson_id)
                                         ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_lesson_student_student FOREIGN KEY (student_id) REFERENCES student (student_id)
                                         ON DELETE RESTRICT ON UPDATE CASCADE
);

-- ---------------------------------------------------------------------
-- Рівень 3: посилаються на запис на заняття (складений FK)
-- ---------------------------------------------------------------------
CREATE TABLE attendance (
    attendance_id    UUID          NOT NULL DEFAULT gen_random_uuid(),
    lesson_id        UUID          NOT NULL,
    student_id       UUID          NOT NULL,
    status           VARCHAR(10)   NOT NULL,
    payment_method   VARCHAR(15),
    amount           NUMERIC(10,2) NOT NULL DEFAULT 0,
    discount_percent NUMERIC(5,2)  NOT NULL DEFAULT 0,
    subscription_id  UUID,
```

```
    print_enabled    BOOLEAN       NOT NULL DEFAULT FALSE,
    print_price      NUMERIC(10,2),
    marked_by        UUID,
    marked_at        TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CONSTRAINT pk_attendance                PRIMARY KEY (attendance_id),
    CONSTRAINT uq_attendance_lesson_student UNIQUE (lesson_id, student_id),
    CONSTRAINT fk_attendance_lesson_student FOREIGN KEY (lesson_id, student_id)
                                            REFERENCES lesson_student (lesson_id, student_id)
                                            ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_attendance_subscription   FOREIGN KEY (subscription_id)
                                            REFERENCES subscription (subscription_id)
                                            ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_attendance_employee       FOREIGN KEY (marked_by) REFERENCES employee
(employee_id)
                                            ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT chk_attendance_status        CHECK (status IN ('present', 'absent', 'excused')),
    CONSTRAINT chk_attendance_payment       CHECK (payment_method IN
                                                   ('card', 'cash', 'subscription',
'certificate')),
    CONSTRAINT chk_attendance_money         CHECK (amount >= 0 AND print_price >= 0
                                                    AND discount_percent BETWEEN 0 AND 100),
    CONSTRAINT chk_attendance_only_present  CHECK (status = 'present'
                                                   OR (payment_method IS NULL AND amount = 0)),
    CONSTRAINT chk_attendance_subscription  CHECK (subscription_id IS NULL
                                                   OR payment_method = 'subscription')
);

CREATE TABLE work (
    work_id       UUID         NOT NULL DEFAULT gen_random_uuid(),
    lesson_id     UUID         NOT NULL,
    student_id    UUID         NOT NULL,
    photo_url     TEXT,
    is_trial      BOOLEAN      NOT NULL DEFAULT FALSE,
    printed       BOOLEAN      NOT NULL DEFAULT FALSE,
    printed_by    UUID,
    printed_at    TIMESTAMPTZ,
    printer_id    SMALLINT,
    filament_id   SMALLINT,
    weight_g      NUMERIC(7,1),
    print_minutes SMALLINT,
    CONSTRAINT pk_work                PRIMARY KEY (work_id),
    CONSTRAINT uq_work_lesson_student UNIQUE (lesson_id, student_id),
    CONSTRAINT fk_work_lesson_student FOREIGN KEY (lesson_id, student_id)
                                      REFERENCES lesson_student (lesson_id, student_id)
                                      ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_work_employee       FOREIGN KEY (printed_by) REFERENCES employee (employee_id)
                                      ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_work_printer        FOREIGN KEY (printer_id) REFERENCES printer (printer_id)
                                      ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_work_filament       FOREIGN KEY (filament_id) REFERENCES filament (filament_id)
                                      ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT chk_work_print_inputs  CHECK (weight_g >= 0 AND print_minutes >= 0),
    CONSTRAINT chk_work_printed_by    CHECK (printed OR (printed_by IS NULL AND printed_at IS
NULL))
);

-- ---------------------------------------------------------------------
-- Рівень 4: собівартість друку (1:1 з work)
-- ---------------------------------------------------------------------
CREATE TABLE work_cost (
    work_id               UUID          NOT NULL,
    price_per_kg          NUMERIC(8,2),
    waste_percent         NUMERIC(5,2)  NOT NULL DEFAULT 0,
    power_watts           NUMERIC(6,1),
    electricity_tariff    NUMERIC(6,2),
    depreciation_per_hour NUMERIC(8,2),
    material_cost         NUMERIC(10,2) NOT NULL DEFAULT 0,
    electricity_cost      NUMERIC(10,2) NOT NULL DEFAULT 0,
    depreciation_cost     NUMERIC(10,2) NOT NULL DEFAULT 0,
    computed_at           TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CONSTRAINT pk_work_cost        PRIMARY KEY (work_id),
    CONSTRAINT fk_work_cost_work   FOREIGN KEY (work_id) REFERENCES work (work_id)
                                   ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT chk_work_cost_input CHECK (price_per_kg >= 0 AND power_watts >= 0
                                          AND electricity_tariff >= 0 AND depreciation_per_hour
>= 0
                                          AND waste_percent BETWEEN 0 AND 100),
    CONSTRAINT chk_work_cost_money CHECK (material_cost >= 0 AND electricity_cost >= 0
                                          AND depreciation_cost >= 0)
);
```

ДОДАТОК Б. Скрипт наповнення таблиць

Файл `docs/db-labs/lab3/sql/02_insert.sql`.

```
-- ============================================================================
-- Лабораторна робота №3. Крок 3: наповнення таблиць (INSERT)
-- Імена, телефони й адреси — вигадані. Курси — справжні курси студії.
-- Стовпці перелiковано явно; первинні ключі IDENTITY та UUID не
-- вказуються — їх генерує СУБД.
-- ============================================================================

-- ----------------------------------------------------------------------------
-- Співробітники (10)
-- ----------------------------------------------------------------------------
INSERT INTO employee (email, last_name, first_name, role, active) VALUES
    ('director@stemik.test',  'Коваленко', 'Ірина',    'superadmin', TRUE),
    ('admin@stemik.test',     'Мельник',   'Олег',     'admin',      TRUE),
    ('admin2@stemik.test',    'Литвин',    'Юлія',     'admin',      TRUE),
    ('shevchuk@stemik.test',  'Шевчук',    'Андрій',   'teacher',    TRUE),
    ('bondar@stemik.test',    'Бондар',    'Марія',    'teacher',    TRUE),
    ('tkachenko@stemik.test', 'Ткаченко',  'Дмитро',   'teacher',    TRUE),
    ('kravets@stemik.test',   'Кравець',   'Олена',    'teacher',    TRUE),
    ('polishchuk@stemik.test','Поліщук',   'Ігор',     'teacher',    TRUE),
    ('moroz@stemik.test',     'Мороз',     'Катерина', 'teacher',    TRUE),
    ('savchuk@stemik.test',   'Савчук',    'Віктор',   'teacher',    FALSE);

-- ----------------------------------------------------------------------------
-- Курси (9 — увесь перелік курсів студії) і тарифи (9 × 6 = 54)
-- ----------------------------------------------------------------------------
INSERT INTO course (name, direction, age_min, age_max, online_print_price) VALUES
    ('3D Tinkercad FIRST (9-12)', '3D',    9, 12, 100),
    ('3D Tinkercad PRO (9-12)',   '3D',    9, 12, 100),
    ('3D Makers FIRST (6-8)',     '3D',    6,  8, 100),
    ('3D Makers PRO (6-8)',       '3D',    6,  8, 100),
    ('3D слайсинг',               '3D',   10, 14, 100),
    ('VR ІГРОВИЙ',                'VR',    7, 12, NULL),
    ('VR РОЗРОБКА (Delightex)',   'VR',    9, 14, NULL),
    ('Robo AI (Ukit 6-8)',        'ROBO',  6,  8, NULL),
    ('Robo AI (UGot 9-12)',       'ROBO',  9, 12, NULL);

-- Один INSERT ... SELECT: кожен курс × кожен тип тарифу
INSERT INTO tariff (course_id, tariff_type, teacher_pay, lesson_price, subscription_price)
SELECT c.course_id, t.tariff_type, t.pay, t.price,
       CASE WHEN t.sub IS NOT NULL THEN t.sub + (c.course_id % 3) * 40 END
FROM course AS c
CROSS JOIN (VALUES
    ('group_1',           250, 450, 1600),
    ('group_2_6',         350, 400, 1440),
    ('individual',        300, 600, 2200),
    ('individual_online', 250, 500, 1800),
    ('group_online',      300, 350, 1200),
    ('extra_service',      50, 150, NULL)
) AS t (tariff_type, pay, price, sub);

-- ----------------------------------------------------------------------------
-- Обладнання й матеріали для 3D-друку (довідники)
-- ----------------------------------------------------------------------------
INSERT INTO printer (name, power_watts, depreciation_per_hour) VALUES
    ('Bambu Lab P1S',   350, 6.00),
    ('Creality K1',     300, 4.00),
    ('Prusa MK4',       240, 5.00);

INSERT INTO filament (material, color, price_per_kg) VALUES
    ('PLA',  'білий',    650),
    ('PLA',  'червоний', 650),
    ('PLA',  'синій',    680),
    ('PETG', 'чорний',   780),
    ('PETG', 'прозорий', 820),
    ('TPU',  'жовтий',   1150);

-- ----------------------------------------------------------------------------
-- Батьки (12), паролі (8), сесії входу (6)
-- ----------------------------------------------------------------------------
INSERT INTO parent (phone, last_name, first_name, last_login_at) VALUES
    ('380501112233', 'Литвиненко', 'Олена',    '2026-11-01 19:05+02'),
    ('380671234567', 'Литвиненко', 'Андрій',   NULL),
    ('380931002030', 'Савченко',   'Наталія',  '2026-11-02 08:40+02'),
    ('380661234000', 'Кравчук',    'Сергій',   NULL),
```

```
    ('380962223344', 'Гончаренко', 'Тетяна',   '2026-10-30 21:15+02'),
    ('380503334455', 'Бойко',      'Ольга',    '2026-10-28 18:00+02'),
    ('380674445566', 'Шевченко',   'Ірина',    '2026-11-02 20:30+02'),
    ('380935556677', 'Романенко',  'Павло',    NULL),
    ('380686667788', 'Олійник',    'Світлана', '2026-10-25 12:10+02'),
    ('380997778899', 'Ковальчук',  'Роман',    NULL),
    ('380638889900', 'Ткачук',     'Людмила',  '2026-11-01 10:00+02'),
    ('380739990011', 'Павленко',   'Оксана',   '2026-10-31 17:45+02');

-- Пароль є лише в тих, хто вже входив у кабінет (хеш — демонстраційний)
INSERT INTO parent_credential (parent_id, password_hash, password_set_at)
SELECT parent_id, '$2a$10$demo.hash.' || right(phone, 4), last_login_at - interval '1 day'
FROM parent
WHERE last_login_at IS NOT NULL;

-- Активні сесії: зберігається лише SHA-256 токена
INSERT INTO parent_session (token_hash, parent_id, client, created_at, last_seen_at, expires_at)
SELECT encode(sha256(convert_to('demo-token-' || p.phone, 'UTF8')), 'hex'),
       p.parent_id, v.client, p.last_login_at, p.last_login_at + interval '20 minutes',
       p.last_login_at + interval '30 days'
FROM parent AS p
JOIN (VALUES ('380501112233', 'android'), ('380931002030', 'web'),
             ('380962223344', 'ios'),     ('380674445566', 'android'),
             ('380638889900', 'web'),     ('380739990011', 'macos')) AS v (phone, client)
  ON v.phone = p.phone;

-- ---------------------------------------------------------------------
-- Учні (15) і зв'язок батьки — діти (14)
-- ---------------------------------------------------------------------
INSERT INTO student (last_name, first_name, birthday, phone, notes, created_by)
SELECT v.last_name, v.first_name, v.birthday::date, v.phone, v.notes,
       (SELECT employee_id FROM employee WHERE email = 'admin@stemik.test')
FROM (VALUES
    ('Литвиненко', 'Максим',     '2015-03-14', '+380501112233', 'Любить роботів'),
    ('Литвиненко', 'Софія',      '2018-09-02', '+380501112233', NULL),
    ('Савченко',   'Артем',      '2014-11-21', '+380931002030', 'Алергія на арахіс'),
    ('Кравчук',    'Вероніка',   '2016-05-30', '+380661234000', NULL),
    ('Гончаренко', 'Тимофій',    '2013-01-08', '+380962223344', 'Прийшов на пробне заняття'),
    ('Бойко',      'Марко',      '2017-07-19', '+380503334455', NULL),
    ('Шевченко',   'Злата',      '2015-12-03', '+380674445566', NULL),
    ('Романенко',  'Остап',      '2014-04-25', '+380935556677', 'Брат у групі VR'),
    ('Олійник',    'Дарина',     '2018-02-11', '+380686667788', NULL),
    ('Ковальчук',  'Назар',      '2016-10-07', '+380997778899', NULL),
    ('Ткачук',     'Мирослава',  '2013-06-16', '+380638889900', NULL),
    ('Павленко',   'Богдан',     '2017-03-29', '+380739990011', NULL),
    ('Кузьменко',  'Ева',        '2015-08-22', '+380679876543', NULL),
    ('Гаврилюк',   'Лев',        '2014-01-30', '+380507654321', 'Ліва рука'),
    ('Марченко',   'Аліна',      '2016-09-14', '+380631237890', NULL)
) AS v (last_name, first_name, birthday, phone, notes);

INSERT INTO parent_student (parent_id, student_id, relation)
SELECT p.parent_id, s.student_id,
       CASE WHEN p.first_name IN ('Андрій', 'Сергій', 'Павло', 'Роман') THEN 'father' ELSE
'mother' END
FROM parent AS p
JOIN student AS s ON s.last_name = p.last_name;

-- ---------------------------------------------------------------------
-- Заняття (15): два тижні жовтня і початок листопада
-- ---------------------------------------------------------------------
INSERT INTO lesson (course_id, teacher_id, lesson_date, start_time, duration_minutes,
                    format, room, theme, completed)
SELECT c.course_id, e.employee_id, v.d::date, v.t::time, v.dur, v.fmt, v.room, v.theme,
       v.d::date < DATE '2026-11-03'
FROM (VALUES
    ('3D Tinkercad FIRST (9-12)', 'shevchuk',  '2026-10-12', '16:00', 60, 'group',
'Клас 3D',            'Брелок з ініціалами'),
    ('Robo AI (Ukit 6-8)',        'tkachenko', '2026-10-13', '17:00', 60, 'group',
'Клас AI',            'Робот-сортувальник'),
    ('VR РОЗРОБКА (Delightex)',   'bondar',    '2026-10-14', '15:30', 90, 'individual',
'Клас VR',            'Перша VR-сцена'),
    ('3D Makers FIRST (6-8)',     'shevchuk',  '2026-10-15', '14:00', 45, 'individual_online',
NULL,                 'Іграшка-динозавр'),
    ('3D Tinkercad FIRST (9-12)', 'shevchuk',  '2026-10-19', '16:00', 60, 'group',
'Клас 3D',            'Підставка для телефону'),
    ('Robo AI (Ukit 6-8)',        'tkachenko', '2026-10-20', '17:00', 60, 'group',
'Клас AI',            'Датчик кольору'),
    ('VR ІГРОВИЙ',                'bondar',    '2026-10-21', '16:00', 60, 'group',
'Клас VR',            'Квест у віртуальному музеї'),
```

```
    ('Robo AI (UGot 9-12)',       'tkachenko', '2026-10-22', '18:00', 60, 'group',
'Комп''ютерний клас', 'Лінійний датчик'),
    ('3D Tinkercad FIRST (9-12)', 'shevchuk',  '2026-10-26', '16:00', 60, 'group',
'Клас 3D',            'Органайзер для олівців'),
    ('3D Makers PRO (6-8)',       'kravets',   '2026-10-27', '17:00', 60, 'group',
'Клас 3D',            'Будиночок для фігурок'),
    ('3D Tinkercad PRO (9-12)',   'kravets',   '2026-10-28', '15:00', 60, 'individual',
'Клас 3D',            'Шестерні та механізми'),
    ('3D слайсинг',               'shevchuk',  '2026-10-29', '16:30', 60, 'group_online',
NULL,                 'Налаштування підтримок'),
    ('3D Tinkercad FIRST (9-12)', 'shevchuk',  '2026-11-02', '16:00', 60, 'group',
'Клас 3D',            'Іменна табличка'),
    ('Robo AI (UGot 9-12)',       'tkachenko', '2026-11-05', '17:00', 60, 'group',
'Комп''ютерний клас', 'Змагання роботів'),
    ('VR РОЗРОБКА (Delightex)',   'bondar',    '2026-11-06', '15:30', 90, 'individual',
'Клас VR',            'Анімація об''єктів')
) AS v (course, teacher, d, t, dur, fmt, room, theme)
JOIN course   AS c ON c.name  = v.course
JOIN employee AS e ON e.email = v.teacher || '@stemik.test';

-- Заміна: 2 листопада заняття Шевчука веде Кравець
UPDATE lesson
SET substitute_id = (SELECT employee_id FROM employee WHERE email = 'kravets@stemik.test')
WHERE lesson_date = '2026-11-02';

-- ---------------------------------------------------------------------
-- Запис учнів на заняття (48)
-- ---------------------------------------------------------------------
INSERT INTO lesson_student (lesson_id, student_id)
SELECT l.lesson_id, s.student_id
FROM (VALUES
    ('Брелок з ініціалами',        'Максим,Артем,Тимофій,Остап,Лев'),
    ('Робот-сортувальник',         'Софія,Вероніка,Марко,Дарина'),
    ('Перша VR-сцена',             'Артем'),
    ('Іграшка-динозавр',           'Софія'),
    ('Підставка для телефону',     'Максим,Артем,Остап,Лев,Злата'),
    ('Датчик кольору',             'Софія,Вероніка,Марко,Назар'),
    ('Квест у віртуальному музеї', 'Тимофій,Мирослава,Ева,Злата'),
    ('Лінійний датчик',            'Остап,Мирослава,Лев'),
    ('Органайзер для олівців',     'Максим,Артем,Злата,Ева'),
    ('Будиночок для фігурок',      'Марко,Дарина,Богдан,Аліна'),
    ('Шестерні та механізми',      'Тимофій'),
    ('Налаштування підтримок',     'Мирослава,Ева,Лев'),
    ('Іменна табличка',            'Максим,Артем,Остап,Злата,Ева'),
    ('Змагання роботів',           'Остап,Мирослава,Лев'),
    ('Анімація об''єктів',         'Артем')
) AS v (theme, names)
CROSS JOIN LATERAL unnest(string_to_array(v.names, ',')) AS n (first_name)
JOIN lesson  AS l ON l.theme = v.theme
JOIN student AS s ON s.first_name = n.first_name;

-- «Заморожені» ЗП і ціна заняття за тарифом: група з одного учня —
-- тариф group_1, з кількох — group_2_6, інші формати — свій тариф
UPDATE lesson AS l
SET payroll_amount = t.teacher_pay,
    price_amount   = t.lesson_price
FROM tariff AS t
WHERE t.course_id = l.course_id
  AND t.tariff_type = CASE
        WHEN l.format <> 'group' THEN l.format
        WHEN (SELECT count(*) FROM lesson_student ls WHERE ls.lesson_id = l.lesson_id) > 1
             THEN 'group_2_6'
        ELSE 'group_1'
      END;

-- ---------------------------------------------------------------------
-- Абонементи (10). Залишок занять перераховується в кроці 5 (UPDATE).
-- ---------------------------------------------------------------------
INSERT INTO subscription (student_id, course_id, total_price, payment_method, purchased_at,
purchased_by)
SELECT s.student_id, c.course_id, t.subscription_price, v.pm, v.d::date,
       (SELECT employee_id FROM employee WHERE email = 'admin@stemik.test')
FROM (VALUES
    ('Максим',    '3D Tinkercad FIRST (9-12)', 'card', '2026-10-12'),
    ('Злата',     '3D Tinkercad FIRST (9-12)', 'cash', '2026-10-19'),
    ('Лев',       '3D Tinkercad FIRST (9-12)', 'card', '2026-10-12'),
    ('Ева',       '3D Tinkercad FIRST (9-12)', 'card', '2026-10-26'),
    ('Софія',     'Robo AI (Ukit 6-8)',        'cash', '2026-10-13'),
    ('Вероніка',  'Robo AI (Ukit 6-8)',        'card', '2026-10-13'),
    ('Марко',     'Robo AI (Ukit 6-8)',        'card', '2026-10-13'),
```

```
    ('Остап',     'Robo AI (UGot 9-12)',       'cash', '2026-10-22'),
    ('Мирослава', 'VR ІГРОВИЙ',                'card', '2026-10-21'),
    ('Артем',     'VR РОЗРОБКА (Delightex)',   'card', '2026-10-14')
) AS v (first_name, course, pm, d)
JOIN student AS s ON s.first_name = v.first_name
JOIN course  AS c ON c.name = v.course
JOIN tariff  AS t ON t.course_id = c.course_id AND t.tariff_type = 'group_2_6';

-- ---------------------------------------------------------------------
-- Відвідування (44 — для всіх проведених занять)
-- Статус: винятки зі списку, решта — «присутній».
-- Оплата: абонемент, якщо він є на цей курс; інакше — по колу
-- картка / готівка / картка / сертифікат (залишок від ділення номера рядка).
-- ---------------------------------------------------------------------
INSERT INTO attendance (lesson_id, student_id, status, payment_method, amount,
                        discount_percent, subscription_id, marked_by, marked_at)
SELECT x.lesson_id, x.student_id, x.status,
       x.pm,
       CASE WHEN x.pm IN ('card', 'cash') THEN x.price_amount ELSE 0 END,
       CASE WHEN x.pm = 'certificate' THEN 100 ELSE 0 END,
       CASE WHEN x.pm = 'subscription' THEN x.sub_id END,
       (SELECT employee_id FROM employee WHERE email = 'admin@stemik.test'),
       x.lesson_date + time '20:00'
FROM (
    SELECT ls.lesson_id, ls.student_id, l.lesson_date, l.price_amount, sb.subscription_id AS
sub_id,
           COALESCE(ex.status, 'present') AS status,
           CASE
               WHEN COALESCE(ex.status, 'present') <> 'present' THEN NULL
               WHEN ex.status IS NULL AND s.first_name = 'Аліна' THEN NULL   -- борг: не оплачено
               WHEN sb.subscription_id IS NOT NULL THEN 'subscription'
               ELSE (ARRAY['card', 'cash', 'card', 'certificate'])
                    [row_number() OVER (ORDER BY l.lesson_date, s.last_name) % 4 + 1]
           END AS pm
    FROM lesson_student AS ls
    JOIN lesson  AS l ON l.lesson_id = ls.lesson_id
    JOIN student AS s ON s.student_id = ls.student_id
    LEFT JOIN subscription AS sb
           ON sb.student_id = ls.student_id AND sb.course_id = l.course_id
    LEFT JOIN (VALUES
        ('Робот-сортувальник',         'Дарина', 'absent'),
        ('Датчик кольору',             'Назар',  'absent'),
        ('Квест у віртуальному музеї', 'Злата',  'excused'),
        ('Органайзер для олівців',     'Ева',    'excused'),
        ('Іменна табличка',            'Артем',  'absent')
    ) AS ex (theme, first_name, status)
           ON ex.theme = l.theme AND ex.first_name = s.first_name
    WHERE l.completed
) AS x;

-- ---------------------------------------------------------------------
-- Роботи учнів (13) і собівартість друку надрукованих
-- ---------------------------------------------------------------------
INSERT INTO work (lesson_id, student_id, photo_url, is_trial, printed, printed_by, printed_at,
                  printer_id, filament_id, weight_g, print_minutes)
SELECT l.lesson_id, s.student_id,
       'works-photos/' || to_char(l.lesson_date, 'YYYYMMDD') || '-' || s.first_name || '.jpg',
       v.trial, v.printer IS NOT NULL,
       CASE WHEN v.printer IS NOT NULL THEN (SELECT employee_id FROM employee WHERE email =
'shevchuk@stemik.test') END,
       CASE WHEN v.printer IS NOT NULL THEN l.lesson_date + time '19:30' END,
       p.printer_id, f.filament_id, v.w, v.m
FROM (VALUES
    ('Брелок з ініціалами',     'Максим',  FALSE, 'Bambu Lab P1S', 'червоний', 12.5, 48),
    ('Брелок з ініціалами',     'Артем',   FALSE, 'Creality K1',   'білий',     9.0, 35),
    ('Брелок з ініціалами',     'Тимофій', TRUE,  'Bambu Lab P1S', 'синій',    10.2, 40),
    ('Брелок з ініціалами',     'Остап',   FALSE, 'Prusa MK4',     'білий',    11.4, 44),
    ('Брелок з ініціалами',     'Лев',     FALSE, NULL,            NULL,       NULL, NULL),
    ('Підставка для телефону', 'Злата',   FALSE, 'Bambu Lab P1S', 'синій',    38.0, 95),
    ('Підставка для телефону', 'Лев',     FALSE, 'Creality K1',   'червоний', 41.5, 110),
    ('Органайзер для олівців', 'Ева',     FALSE, 'Prusa MK4',     'білий',    55.0, 140),
    ('Органайзер для олівців', 'Максим',  FALSE, NULL,            NULL,       NULL, NULL),
    ('Будиночок для фігурок',  'Марко',   FALSE, 'Bambu Lab P1S', 'червоний', 63.2, 155),
    ('Будиночок для фігурок',  'Богдан',  FALSE, NULL,            NULL,       NULL, NULL),
    ('Шестерні та механізми',  'Тимофій', FALSE, 'Creality K1',   'синій',    27.8, 70),
    ('Іменна табличка',        'Злата',   FALSE, NULL,            NULL,       NULL, NULL)
) AS v (theme, first_name, trial, printer, color, w, m)
JOIN lesson  AS l ON l.theme = v.theme
JOIN student AS s ON s.first_name = v.first_name
LEFT JOIN printer  AS p ON p.name = v.printer
```

```
LEFT JOIN filament AS f ON f.material = 'PLA' AND f.color = v.color;

INSERT INTO studio_settings (calc_start_date, electricity_tariff, filament_waste_percent,
updated_by)
VALUES ('2026-09-01', 4.32, 10, (SELECT employee_id FROM employee WHERE role = 'superadmin'));

-- матеріал = вага × (1 + відходи) × ціна/кг; електроенергія = кВт × год × тариф;
-- амортизація = год × ставка. Ціни беруться на момент друку (знімок).
INSERT INTO work_cost (work_id, price_per_kg, waste_percent, power_watts, electricity_tariff,
                       depreciation_per_hour, material_cost, electricity_cost, depreciation_cost)
SELECT w.work_id, f.price_per_kg, st.filament_waste_percent, p.power_watts,
st.electricity_tariff,
       p.depreciation_per_hour,
       round(w.weight_g / 1000 * (1 + st.filament_waste_percent / 100) * f.price_per_kg, 2),
       round(p.power_watts / 1000 * w.print_minutes / 60 * st.electricity_tariff, 2),
       round(w.print_minutes / 60.0 * p.depreciation_per_hour, 2)
FROM work AS w
JOIN printer  AS p ON p.printer_id  = w.printer_id
JOIN filament AS f ON f.filament_id = w.filament_id
CROSS JOIN studio_settings AS st
WHERE w.printed;

-- ---------------------------------------------------------------------
-- Коментарі викладачів (10)
-- ---------------------------------------------------------------------
INSERT INTO student_comment (student_id, author_id, body, created_at)
SELECT s.student_id, e.employee_id, v.body, v.at::timestamptz
FROM (VALUES
    ('Максим',    'shevchuk',  'Швидко освоїв Tinkercad, можна давати складніші моделі', '2026-
10-19 18:00+03'),
    ('Артем',     'bondar',    'Потрібна допомога з логікою в CoBlocks',                  '2026-
10-14 17:30+03'),
    ('Софія',     'tkachenko', 'Перевести в групу 9-12 з наступного семестру',            '2026-
10-20 18:30+03'),
    ('Тимофій',   'shevchuk',  'Після пробного заняття записався на індивідуальні',       '2026-
10-12 18:10+03'),
    ('Злата',     'bondar',    'Пропустила квест через хворобу, домовились на відпрацювання',
'2026-10-21 17:20+03'),
    ('Остап',     'tkachenko', 'Лідер команди на змаганнях',                              '2026-
10-22 19:15+03'),
    ('Марко',     'kravets',   'Акуратно працює з дрібними деталями',                     '2026-
10-27 18:20+03'),
    ('Ева',       'shevchuk',  'Цікавиться слайсингом, порадити курс 3D слайсинг',        '2026-
10-26 17:40+03'),
    ('Аліна',     'kravets',   'Батьки обіцяли оплатити заняття 27.10 наступного разу',   '2026-
10-27 18:25+03'),
    ('Мирослава', 'shevchuk',  'Добре налаштовує підтримки, допомагає іншим',             '2026-
10-29 17:45+03')
) AS v (first_name, teacher, body, at)
JOIN student  AS s ON s.first_name = v.first_name
JOIN employee AS e ON e.email = v.teacher || '@stemik.test';
```

ДОДАТОК В. Скрипти ALTER, UPDATE/DELETE, TRUNCATE, DROP і генерування даних

Файл `docs/db-labs/lab3/sql/03_alter.sql`.

```
-- ============================================================
-- Лабораторна робота №3. Крок 4: зміна структури таблиць (ALTER TABLE)
-- Виконується на вже заповнених таблицях, щоб показати вплив на дані.
-- Команди, що мають завершитися помилкою, позначено «очікується помилка».
-- ============================================================
\set ON_ERROR_STOP off
\set VERBOSITY terse

\echo '--- 1. ADD COLUMN без DEFAULT: наявні рядки отримують NULL'
ALTER TABLE student ADD COLUMN school VARCHAR(100);
SELECT count(*) AS students, count(school) AS with_school FROM student;

\echo '--- 2. ADD COLUMN NOT NULL без DEFAULT до заповненої таблиці (очікується помилка)'
ALTER TABLE employee ADD COLUMN hire_date DATE NOT NULL;

\echo '--- 3. ADD COLUMN NOT NULL разом із DEFAULT: значення отримують усі рядки'
ALTER TABLE employee ADD COLUMN hire_date DATE NOT NULL DEFAULT DATE '2025-09-01';
SELECT last_name, hire_date FROM employee ORDER BY last_name LIMIT 4;

\echo '--- 4. ALTER COLUMN TYPE: розширення VARCHAR(20) → VARCHAR(30) безпечне'
ALTER TABLE student ALTER COLUMN phone TYPE VARCHAR(30);

\echo '--- 5. ALTER COLUMN TYPE: звуження VARCHAR(60) → VARCHAR(10) (очікується помилка)'
ALTER TABLE course ALTER COLUMN name TYPE VARCHAR(10);

\echo '--- 6. ALTER COLUMN TYPE з USING: тривалість SMALLINT → INTEGER'
ALTER TABLE lesson ALTER COLUMN duration_minutes TYPE INTEGER USING duration_minutes::integer;
SELECT column_name, data_type, character_maximum_length
FROM information_schema.columns
WHERE (table_name, column_name) IN (('student', 'phone'), ('lesson', 'duration_minutes'),
                                    ('student', 'school'), ('employee', 'hire_date'))
ORDER BY 1;

\echo '--- 7. ADD CONSTRAINT: формат телефону учня +380XXXXXXXXX'
ALTER TABLE student ADD CONSTRAINT chk_student_phone_format CHECK (phone ~ '^\+380[0-9]{9}$');

\echo '--- 8. Перевірка нового обмеження (очікується помилка)'
UPDATE student SET phone = '067 123 45 67' WHERE first_name = 'Максим';

\echo '--- 9. Обмеження, яке порушують наявні дані, не додається (очікується помилка)'
ALTER TABLE student ADD CONSTRAINT chk_student_has_notes CHECK (notes IS NOT NULL);

\echo '--- 10. DROP CONSTRAINT за назвою'
ALTER TABLE student DROP CONSTRAINT chk_student_phone_format;
SELECT conname FROM pg_constraint
WHERE conrelid = 'student'::regclass AND contype = 'c'
ORDER BY 1;

\echo '--- 11. RENAME COLUMN і DROP COLUMN: повертаємо таблицю student до моделі'
ALTER TABLE student RENAME COLUMN school TO school_name;
ALTER TABLE student DROP COLUMN school_name;
```

Файл `docs/db-labs/lab3/sql/04_update_delete.sql`.

```
-- ============================================================
-- Лабораторна робота №3. Крок 5: оновлення та видалення (UPDATE, DELETE)
-- Правило: спершу SELECT із тією самою умовою WHERE, потім зміна.
-- ============================================================
\set ON_ERROR_STOP off
\set VERBOSITY terse

\echo '--- 1. UPDATE ... FROM: перерахувати залишок абонементів за відвідуваннями'
\echo '    (спершу SELECT — що саме зміниться)'
SELECT s.first_name AS student, c.name AS course, sb.total_lessons,
       count(a.attendance_id) AS used
FROM subscription AS sb
JOIN student AS s ON s.student_id = sb.student_id
JOIN course  AS c ON c.course_id  = sb.course_id
LEFT JOIN attendance AS a ON a.subscription_id = sb.subscription_id
```

```
GROUP BY s.first_name, c.name, sb.total_lessons
ORDER BY used DESC, student;

UPDATE subscription AS sb
SET remaining_lessons = sb.total_lessons - u.used,
    status = CASE WHEN sb.total_lessons - u.used = 0 THEN 'completed' ELSE 'active' END
FROM (SELECT subscription_id, count(*) AS used
      FROM attendance
      WHERE subscription_id IS NOT NULL
      GROUP BY subscription_id) AS u
WHERE u.subscription_id = sb.subscription_id;

\echo '--- 2. UPDATE з умовою і RETURNING: +10 % до ціни групових 3D-занять'
SELECT c.name, t.tariff_type, t.lesson_price
FROM tariff t JOIN course c USING (course_id)
WHERE c.direction = '3D' AND t.tariff_type = 'group_2_6';

UPDATE tariff AS t
SET lesson_price = round(t.lesson_price * 1.10, 2)
FROM course AS c
WHERE c.course_id = t.course_id
  AND c.direction = '3D'
  AND t.tariff_type = 'group_2_6'
RETURNING c.name, t.tariff_type, t.lesson_price AS new_price;

\echo '--- 3. UPDATE: борг погашено — заняття 27.10 оплачене готівкою'
UPDATE attendance AS a
SET payment_method = 'cash', amount = l.price_amount
FROM lesson AS l, student AS s
WHERE l.lesson_id = a.lesson_id AND s.student_id = a.student_id
  AND a.status = 'present' AND a.payment_method IS NULL
RETURNING s.first_name, l.lesson_date, a.payment_method, a.amount;

\echo '--- 4. DELETE з умовою: коментар про борг більше не актуальний'
SELECT left(body, 50) AS body FROM student_comment WHERE body LIKE '%оплатити%';
DELETE FROM student_comment WHERE body LIKE '%оплатити%';

\echo '--- 5. DELETE ... RETURNING: завершити сесії, неактивні з 31.10'
DELETE FROM parent_session
WHERE last_seen_at < TIMESTAMPTZ '2026-10-31 00:00+02'
RETURNING client, last_seen_at::date AS last_seen;

\echo '--- 6. DELETE батьківського рядка, на який посилаються (RESTRICT, очікується помилка)'
DELETE FROM student WHERE first_name = 'Артем';

\echo '--- 7. DELETE з каскадом (у транзакції з відкатом): кабінет батьків Ковальчука'
BEGIN;
SELECT (SELECT count(*) FROM parent) AS parents, (SELECT count(*) FROM parent_student) AS links;
DELETE FROM parent WHERE last_name = 'Ковальчук';
SELECT (SELECT count(*) FROM parent) AS parents, (SELECT count(*) FROM parent_student) AS links;
ROLLBACK;

\echo '--- 8. Небезпека UPDATE без WHERE: змінено ВСІ рядки (відкат)'
BEGIN;
UPDATE lesson SET room = 'Клас AI';
SELECT room, count(*) FROM lesson GROUP BY room;
ROLLBACK;
SELECT room, count(*) FROM lesson GROUP BY room ORDER BY 2 DESC;
```

Файл `docs/db-labs/lab3/sql/05_truncate.sql`.

```
-- ================================================================
-- Лабораторна робота №3. Крок 6: TRUNCATE TABLE і відмінність від DELETE
-- Усі руйнівні команди — у транзакціях з ROLLBACK: у PostgreSQL
-- TRUNCATE транзакційний і його можна відкотити.
-- ================================================================
\set ON_ERROR_STOP off
\set VERBOSITY terse

\echo '--- 1. DELETE усіх рядків: лічильник IDENTITY продовжує рахунок'
BEGIN;
DELETE FROM tariff;
INSERT INTO tariff (course_id, tariff_type, teacher_pay, lesson_price)
VALUES (1, 'group_1', 250, 450) RETURNING tariff_id;
ROLLBACK;

\echo '--- 2. TRUNCATE без опцій: у PostgreSQL за замовчуванням CONTINUE IDENTITY'
BEGIN;
```

```
TRUNCATE TABLE tariff;
INSERT INTO tariff (course_id, tariff_type, teacher_pay, lesson_price)
VALUES (1, 'group_1', 250, 450) RETURNING tariff_id;
ROLLBACK;

\echo '--- 3. TRUNCATE ... RESTART IDENTITY: лічильник починається з 1'
BEGIN;
TRUNCATE TABLE tariff RESTART IDENTITY;
INSERT INTO tariff (course_id, tariff_type, teacher_pay, lesson_price)
VALUES (1, 'group_1', 250, 450) RETURNING tariff_id;
ROLLBACK;
SELECT count(*) AS tariffs_after_rollback FROM tariff;

\echo '--- 4. TRUNCATE таблиці, на яку посилається FK (очікується помилка)'
TRUNCATE TABLE printer;

\echo '--- 5. DELETE тієї самої таблиці працює: FK перевіряється для кожного рядка (SET NULL)'
BEGIN;
DELETE FROM printer;
SELECT count(*) AS works, count(printer_id) AS works_with_printer FROM work;
ROLLBACK;

\echo '--- 6. TRUNCATE ... CASCADE очищає й усі залежні таблиці!'
BEGIN;
TRUNCATE TABLE printer CASCADE;
SELECT (SELECT count(*) FROM printer) AS printers, (SELECT count(*) FROM work) AS works,
       (SELECT count(*) FROM work_cost) AS work_costs;
ROLLBACK;
SELECT (SELECT count(*) FROM printer) AS printers, (SELECT count(*) FROM work) AS works;

\echo '--- 7. Швидкість на 500 000 рядків: DELETE проти TRUNCATE'
CREATE TEMP TABLE bulk_log AS
SELECT g AS id, now() - g * interval '1 second' AS logged_at, md5(g::text) AS payload
FROM generate_series(1, 500000) AS g;
\timing on
DELETE FROM bulk_log;
\timing off
INSERT INTO bulk_log
SELECT g, now() - g * interval '1 second', md5(g::text) FROM generate_series(1, 500000) AS g;
\timing on
TRUNCATE TABLE bulk_log;
\timing off
DROP TABLE bulk_log;
```

Файл `docs/db-labs/lab3/sql/06_drop.sql`.

```
-- ================================================================
-- Лабораторна робота №3. Крок 7: видалення таблиць (DROP TABLE)
-- DDL у PostgreSQL транзакційний, тому кожен сценарій виконано в
-- транзакції та відкочено — база лишається цілою для ЛР №4.
-- ================================================================
\set ON_ERROR_STOP off
\set VERBOSITY default

\echo '--- 1. DROP батьківської таблиці, на яку посилаються FK (очікується помилка)'
DROP TABLE course;

\echo '--- 2. Які зовнішні ключі тримають course'
SELECT conrelid::regclass AS child_table, conname AS constraint_name
FROM pg_constraint
WHERE confrelid = 'course'::regclass
ORDER BY 1;

\set VERBOSITY terse
\echo '--- 3. Правильний порядок — від дочірніх до батьківських (зворотний до створення)'
BEGIN;
DROP TABLE work_cost;
DROP TABLE attendance, work;
DROP TABLE lesson_student;
DROP TABLE tariff, subscription, lesson;
DROP TABLE course;
SELECT count(*) AS tables_left FROM pg_tables WHERE schemaname = 'public';
ROLLBACK;

\echo '--- 4. DROP ... CASCADE видаляє не дочірні таблиці, а їхні FK'
BEGIN;
DROP TABLE course CASCADE;
SELECT count(*) AS tables_left FROM pg_tables WHERE schemaname = 'public';
```

```
SELECT count(*) AS lessons_still_here FROM lesson;
SELECT count(*) AS fk_to_course FROM pg_constraint WHERE conname LIKE 'fk_%_course';
ROLLBACK;

\echo '--- 5. DROP «листової» таблиці, на яку ніхто не посилається'
BEGIN;
DROP TABLE student_comment;
DROP TABLE IF EXISTS student_comment;
ROLLBACK;
SELECT count(*) AS tables_after_rollback FROM pg_tables WHERE schemaname = 'public';
```

Файл `docs/db-labs/lab3/sql/07_generate.sql`.

```
-- ======================================================================
-- Лабораторна робота №3. Крок 8: масове генерування тестових даних
-- Аналоги прийомів із методичних вказівок (MS SQL Server) у PostgreSQL:
--   WHILE з лічильником       → цикл WHILE у блоці DO (PL/pgSQL)
--   CROSS JOIN sys.objects    → функція generate_series()
--   рекурсивний CTE           → WITH RECURSIVE (без MAXRECURSION)
--   ORDER BY NEWID()          → random(); розподіл — залишок від ділення %
-- Усе виконується в транзакції з ROLLBACK: база лишається з даними кроку 3.
-- ======================================================================
\set ON_ERROR_STOP off
\set VERBOSITY terse

BEGIN;

\echo '--- 1. Цикл WHILE у блоці DO: 100 тестових учнів по одному INSERT'
DO $$
DECLARE
    i INT := 1;
BEGIN
    WHILE i <= 100 LOOP
        INSERT INTO student (last_name, first_name, birthday, phone)
        VALUES ('Тестовий' || i,
                'Учень' || i,
                DATE '2013-01-01' + (i * 37 % 2190),           -- 2013–2018 рр.
                '+38050' || lpad(i::text, 7, '0'));
        i := i + 1;
    END LOOP;
END $$;
SELECT count(*) AS students FROM student;

\echo '--- 2. generate_series: 1000 батьківських кабінетів одним INSERT ... SELECT'
INSERT INTO parent (phone, last_name, first_name)
SELECT '38067' || lpad(seq::text, 7, '0'),
       'Батько' || seq,
       (ARRAY['Олена', 'Андрій', 'Наталія', 'Сергій', 'Ірина'])[seq % 5 + 1]
FROM generate_series(1, 1000) AS seq;
SELECT count(*) AS generated, count(DISTINCT first_name) AS distinct_names,
       min(phone), max(phone)
FROM parent WHERE last_name LIKE 'Батько%';

\echo '--- 3. WITH RECURSIVE: щотижнева серія занять до кінця навчального року'
WITH RECURSIVE week (n, d) AS (
    SELECT 1, DATE '2026-11-09'
    UNION ALL
    SELECT n + 1, d + 7 FROM week WHERE d + 7 <= DATE '2027-05-31'
),
series AS (SELECT gen_random_uuid() AS id)
INSERT INTO lesson (course_id, teacher_id, series_id, lesson_date, start_time, format, room,
theme)
SELECT c.course_id, e.employee_id, series.id, week.d, TIME '16:00', 'group', 'Клас 3D',
       'Тема ' || week.n
FROM week, series,
     course   AS c,
     employee AS e
WHERE c.name = '3D Tinkercad FIRST (9-12)' AND e.email = 'shevchuk@stemik.test';
SELECT count(*) AS lessons_in_series, min(lesson_date), max(lesson_date)
FROM lesson WHERE theme LIKE 'Тема %';

\echo '--- 4. Випадковий вибір і залишок від ділення: розклад на 200 занять'
INSERT INTO lesson (course_id, teacher_id, lesson_date, start_time, format, room, theme)
SELECT (seq % 9) + 1,                                          -- курси 1..9 по колу
       (SELECT employee_id FROM employee
        WHERE role = 'teacher' AND active AND seq > 0           -- seq > 0: підзапит
        ORDER BY random() LIMIT 1),                             --   для кожного рядка
       DATE '2026-12-01' + (seq % 120),
```

```
       TIME '14:00' + (seq % 5) * interval '1 hour',
       'group',
       (ARRAY['Клас AI', 'Клас VR', 'Комп''ютерний клас', 'Клас 3D'])[seq % 4 + 1],
       'Згенероване заняття ' || seq
FROM generate_series(1, 200) AS seq;
SELECT room, count(*) AS lessons, count(DISTINCT teacher_id) AS teachers
FROM lesson WHERE theme LIKE 'Згенероване%'
GROUP BY room ORDER BY room;

ROLLBACK;
SELECT (SELECT count(*) FROM student) AS students, (SELECT count(*) FROM parent) AS parents,
       (SELECT count(*) FROM lesson) AS lessons;

\echo '--- 5. Швидкість: 20 000 рядків циклом WHILE і одним INSERT ... SELECT'
CREATE TEMP TABLE gen_test (n INT, label TEXT);
\timing on
DO $$
DECLARE i INT := 1;
BEGIN
    WHILE i <= 20000 LOOP
        INSERT INTO gen_test VALUES (i, 'row ' || i);
        i := i + 1;
    END LOOP;
END $$;
\timing off
TRUNCATE gen_test;
\timing on
INSERT INTO gen_test SELECT g, 'row ' || g FROM generate_series(1, 20000) AS g;
\timing off
DROP TABLE gen_test;
```